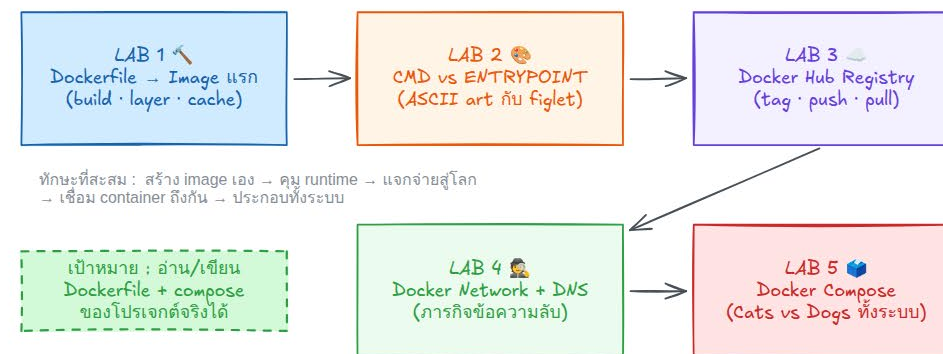


Docker ภาควิชาสร้าง Dockerfile → Compose

build image เอง · ส่งขึ้น Docker Hub · เชื่อม network
ปิดท้ายด้วย docker compose — ลงมือจริง 5 LAB

แผนที่การเดินทาง Week 11 — จาก Dockerfile สู่ระบบเต็มด้วย Compose



ทุกแล็บรันในเครื่องเรียน devtools เดิม (docker ซ่อน docker) — พังแค่ไหนก็ลบทิ้งแล้วเริ่มใหม่ได้เสมอ



| สิ่งที่จะเรียนวันนี้

- ทวน image กับ container

02 CMD vs ENTRYPOINT + **LAB 2**

04 Docker Network + **LAB 4**

06 Multi-stage + Compose Watch

01 Dockerfile & Image + **LAB 1**

03 Docker Registry + **LAB 3**

05 Docker Compose + **LAB 5**

- สรุป + Cleanup

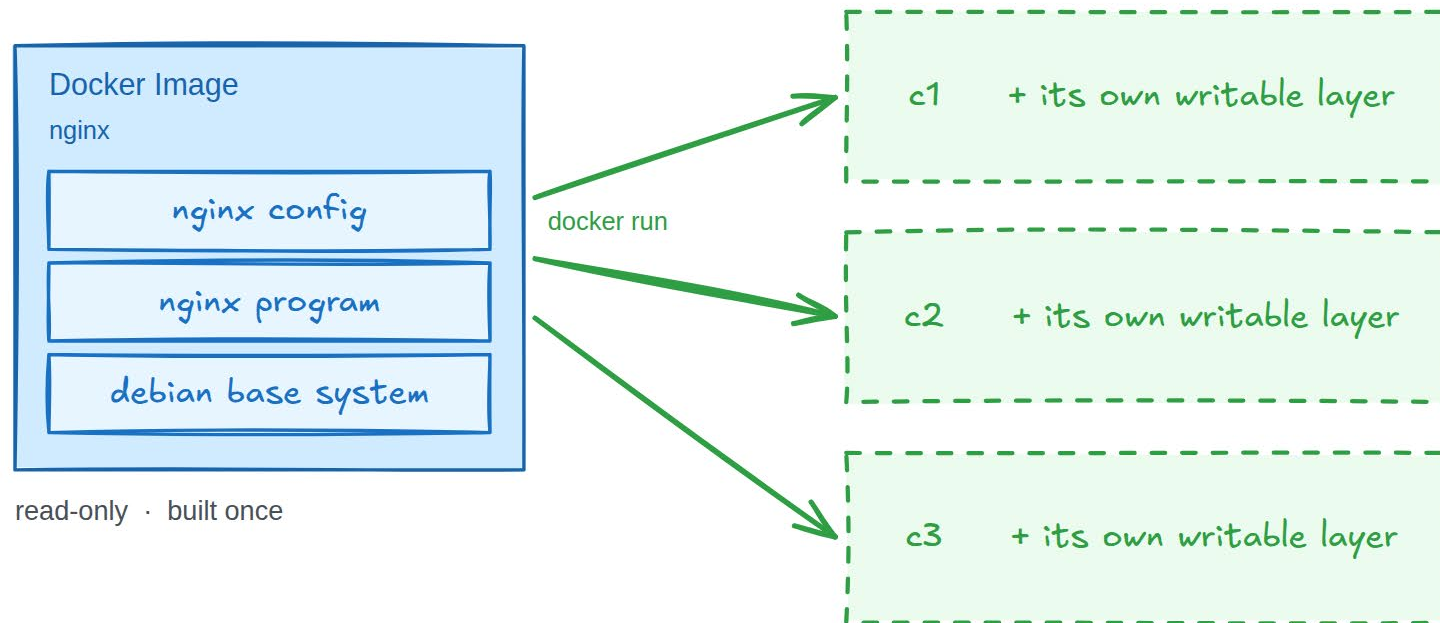
ก่อนเริ่ม 1 นาที: เปิด terminal แล้วเช็ก `docker version` และ `docker compose version` ให้ผ่าน



ทวน : Image คือพิมพ์เขียว · Container คือบ้าน

สัปดาห์ก่อนเรา docker run ของคนอื่น — วันนี้สร้างพิมพ์เขียวเอง

One Image -> Many Containers



The read-only image is SHARED - each container only adds its own thin writable layer

คำถามค้างจากสัปดาห์ก่อน : image อย่าง nginx ที่เรา pull มา **ใครสร้าง? สร้างยังไง?**



วิธีเรียน

ทุก LAB เรียนแบบนักทดลอง

ให้ terminal เป็นกรรมการ

1

Predict — ทาย

ผลจะเป็นแบบไหน?

2

Run — ลงมือ

พิมพ์คำสั่งตาม README

3

Observe — ดู

เทียบ Expected output

4

Explain — อธิบาย

อธิบายไม่ได้ = ยังไม่จบ

ทุกผลลัพธ์มาจาก**การรันจริง**ใน tuchsanai/devtools:2569_1 — เครื่องคุณต้องทำซ้ำได้



01

Dockerfile & Image

จากโค้ด Python สู่ image ที่ย้ายไปรันบนเครื่องที่รองรับ Docker ได้



| ปัญหา : "เครื่องผมรันได้นะ"

ส่งโค้ดให้เพื่อน — ทำไมรันไม่ติด?

Python คนละรุ่น

เรา 3.12 เพื่อน 3.8 — ผลต่างกัน

ไลบรารีไม่ครบ

ไม่รู้ต้อง `pip install` อะไรบ้าง

ตั้งค่าไม่ตรงกัน

OS ต่าง · env ต่าง · แก้ไม่จบ

ทางออก : เขียนสูตรเป็นไฟล์ ให้ Docker ประกอบ**เหมือนกันทุกเครื่อง** — ชื่อ **Dockerfile**



| นิยาม : Dockerfile คืออะไร

ไฟล์ข้อความธรรมดา — สูตรให้ Docker ประกอบ image

- ▶ ระบุ base image · โลบรารี · โค้ด · คำสั่งเริ่ม
- ▶ ผล build คือ **image**
นำไปรันซ้ำหรือส่งต่อได้
- ▶ คำสั่งที่เปลี่ยนไฟล์สร้าง **layer**
Docker นำ layer เดิมกลับมาใช้ตอน build ซ้ำได้
- ▶ เก็บใน git กับโค้ด
ติดตามและ review การเปลี่ยนแปลงได้



| ทายก่อน แล้วค่อยคลิกดูเฉลย

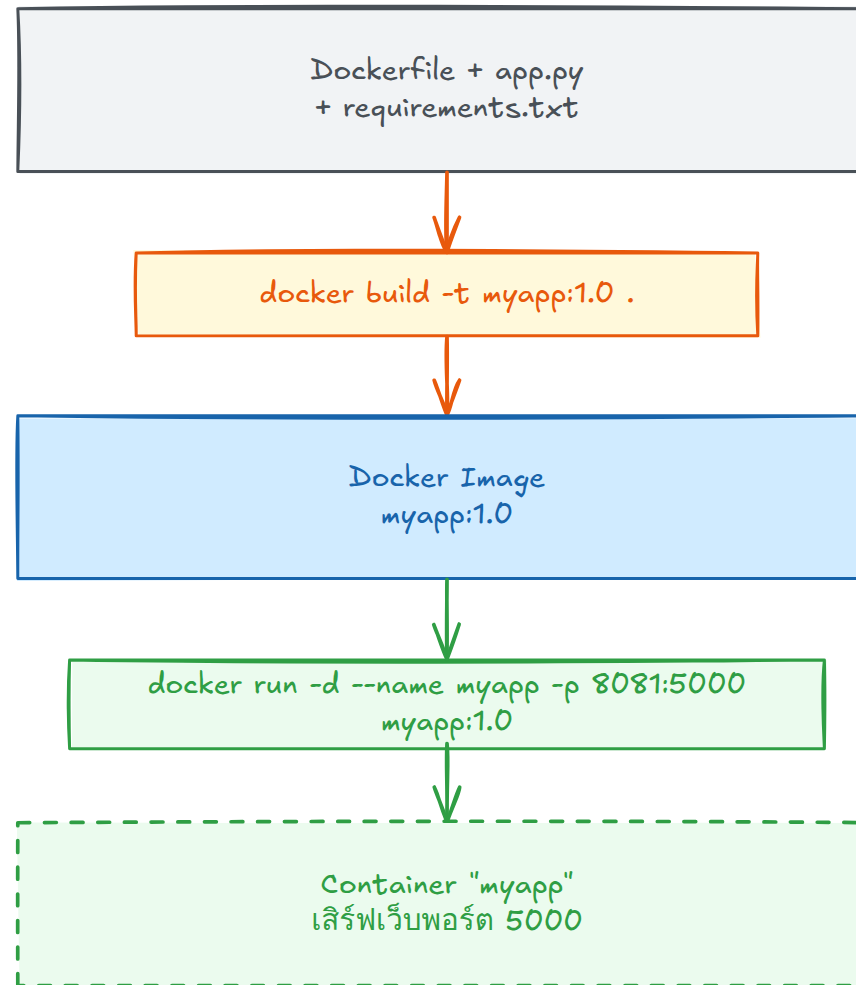
- 1 รัน container 3 ตัวจาก image เดียว — ต้อง build ที่ครั้ง?
- 2 Dockerfile ต้องใช้โปรแกรมพิเศษเปิดไหม?

คลิกเพื่อดูเฉลย

1) ครั้งเดียว — image คือพิมพ์เขียว สร้างบ้านที่หลังก็ได้ · **2)** ไม่ — ไฟล์ข้อความธรรมดา editor ไหนก็เปิดได้



| โดอะแกรม : เส้นทาง Dockerfile → Image → Container



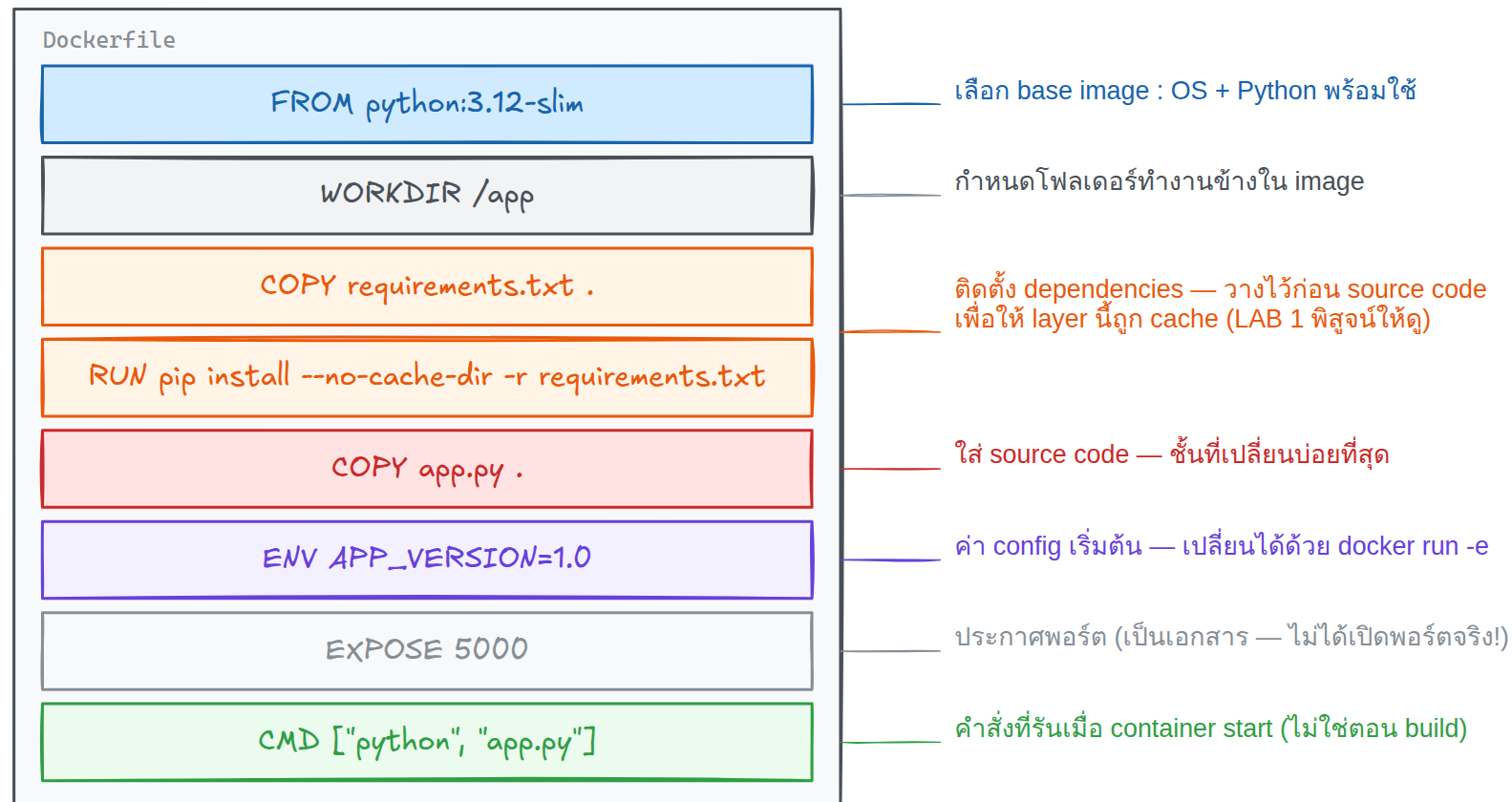
สูตร → พิมพ์เขียว → บ้าน
(build ครั้งเดียว รันก็ container ก็ได้)



อ่าน Dockerfile ของ LAB 1 ทีละบรรทัด

แต่ละบรรทัดตอบคำถามคนละข้อ

กายวิภาคของ Dockerfile (ไฟล์จริงจาก LAB 1)



build จากบนลงล่าง — แต่ละบรรทัดกลายเป็น layer หนึ่งชั้นของ image



คำสั่งที่ทำงาน "ตอน build"

ทุกบรรทัด : **INSTRUCTION** + **ARGUMENT**

คำสั่ง	ทำหน้าที่
<code>FROM python:3.12-slim</code>	เลือก base image — ใช้ tag ระบุรุ่นแทน <code>latest</code>
<code>WORKDIR /app</code>	ตั้งโฟลเดอร์ทำงาน
<code>COPY src dest</code>	คัดลอกเข้า image — ลำดับมีผล (สไลด์ 16)
<code>RUN pip install ...</code>	รันคำสั่ง "อบ" ผลเก็บใน layer

ทำตอน build — ไม่ทำซ้ำเมื่อ `docker run`



ค่าตอนรัน + คำสั่งเริ่มต้น

คำสั่ง	ทำหน้าที่
<code>ENV key=value</code>	ฝังค่า default — กับได้ด้วย <code>-e</code>
<code>EXPOSE 5000</code>	"ป้ายประกาศ" พอร์ต — ไม่ได้เปิดจริง
<code>CMD ["python", "app.py"]</code>	คำสั่งเริ่มต้นตอน start
<code>ENTRYPOINT ["cmd"]</code>	โปรแกรมหลัก (Section 2)
<code>VOLUME</code> · <code>LABEL</code>	จุด mount ข้อมูล · ติด metadata

ตัวเปิดพอร์ตจริงคือ -p — ลืมแล้วเข้าเว็บไม่ได้ทั้งที่ทุกอย่าง Up



| สองจังหวะเวลา ที่คนสับสนบ่อยที่สุด

ตอน build (ครั้งเดียว)

FROM · COPY · RUN — ผลถูก "อบ" ถาวร
ติดตั้ง Flask (ไลบรารีเว็บ Python) ครั้งเดียว

ตอน start (ทุกครั้งที่ run)

CMD / ENTRYPOINT เปิดโปรแกรมหลัก
ENV กับด้วย -e ได้

แก้ไขแล้วรีเฟรช **หน้าเว็บไม่เปลี่ยน** — ต้อง build + run ใหม่



build หรือ start?

- 1 มี EXPOSE แต่ไม่ใส่ -p — เข้าเว็บได้ไหม?
- 2 RUN pip install ทำงานตอนไหน?
- 3 เปลี่ยน ENV โดยไม่ build ใหม่ ได้ไหม?

 **คลิกเพื่อดูเฉลย**

1) ไม่ได้ — ตัวเปิดจริงคือ -p · **2)** ตอน build ครั้งเดียว · **3)** ได้ — -e ชนะ ENV



คำสั่งจริง : docker build

build context = โฟลเดอร์วัตถุที่ส่งให้ Docker — คือความหมายของจุด .

```
$ docker build -t myapp:1.0 .  
#3 [internal] load .dockerignore          ← กันไฟล์ขยะตั้งแต่ต้นทาง  
#4 [internal] load build context — transferring context: 2.42kB  
#5 [1/5] FROM docker.io/library/python:3.12-slim@sha256:229a2c5...  
#6 [2/5] WORKDIR /app  
#7 [3/5] COPY requirements.txt .  
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt  
#9 [5/5] COPY app.py .  
#10 naming to docker.io/library/myapp:1.0 ← image ของเราเกิดแล้ว
```

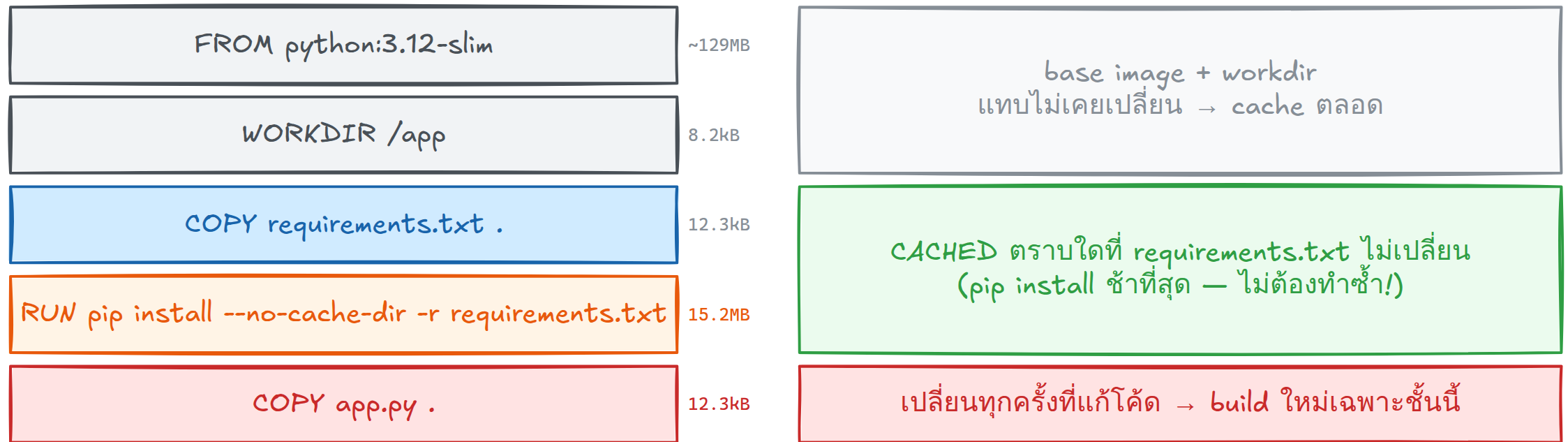
-t ตั้งชื่อ:tag · **จุด** . คือ build context — COPY หยิบได้เฉพาะจากในที่



| โดอะแกรม : Layer & Build Cache

Layer & Build Cache — ทำไมแก้ app.py แล้ว build ใหม่เร็วมาก

กติกา : แก้ layer ไหน → ทุก layer ที่อยู่ "ใต้มันลงไป" ต้อง build ใหม่ทั้งหมด



บทเรียน : วางชั้นที่เปลี่ยนบ่อย (source code) ไว้ "ล่างสุด" → pip install ถูก cache → build รอบถัดไปเหลือแค่ไม่กี่วินาที (ขนาด layer จากผล docker history จริงใน LAB 1 · ส่วน ENV/EXPOSE/CMD เป็น metadata ขนาด 0B)



ผลจริง : build รอบสองเร็วขึ้น ~4 เท่า

แก้ app.py บรรทัดเดียวแล้ว build ใหม่

```
$ time docker build -t myapp:2.0 .
#6 [2/5] WORKDIR /app
#6 CACHED
#7 [3/5] COPY requirements.txt .
#7 CACHED          ← เนื้อไฟล์ไม่เปลี่ยน → ใช้ของเดิม
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED          ← ไม่ติดตั้ง Flask ซ้ำ ไม่แตะเน็ตเลย!
#9 [5/5] COPY app.py .
#9 DONE 0.0s       ← ทำใหม่แค่ขั้นนี้

real    0m1.922s    ← รอบแรก 0m7.924s
```

กติกา cache : แก้ layer ไหน ทุกชั้นใต้มันทำใหม่หมด — ของเปลี่ยนบ่อยไว้ล่างสุด · แกม : build พังกลางทาง สิ่งใหม่ไม่เริ่มศูนย์



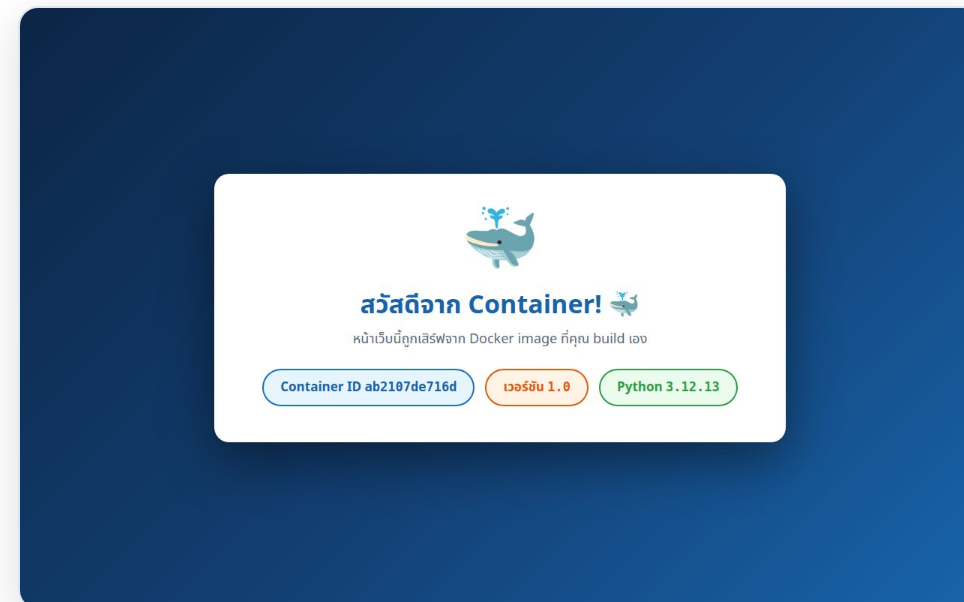
| Error ที่เจอบ่อย ตอน build / run

ข้อความ error	ทางแก้
<code>"docker buildx build" requires exactly 1 argument</code>	ลืมจุด <code>.</code> – เติมท้ายคำสั่ง
<code>failed to read dockerfile ... no such file</code>	โฟลเดอร์ไม่มี Dockerfile – <code>cd</code> ก่อน
<code>name "/myapp" is already in use</code>	ชื่อซ้ำ – <code>docker rm -f myapp</code>
<code>port is already allocated</code>	พอร์ตถูกจอง – ลบตัวจอง/เปลี่ยนพอร์ต

สร้าง Docker Image แรกของคุณ

001_LAB_Dockerfile_First_Image · ~20 นาที · verify.sh

- ▶ build myapp:1.0 · ส่อง docker history
- ▶ -p 8081:5000 · รัน 2 container จาก image เดียว
- ▶ แก้โค้ด → build ใหม่ → เห็น CACHED กับตา
- ▶ กับ ENV ด้วย -e APP_VERSION=2.0
- ▶ พิจารณา .dockerignore : ใน image เหลือ 2 ไฟล์



เว็บจาก image ที่ build เอง — Container ID ตรง docker ps



ทำนายผล build

- 1 แก้เฉพาะ `app.py` แล้ว build ใหม่ — RUN `pip install` รันซ้ำไหม?
- 2 ถ้าใช้ COPY . . บรรทัดเดียว จะเสียอะไร?

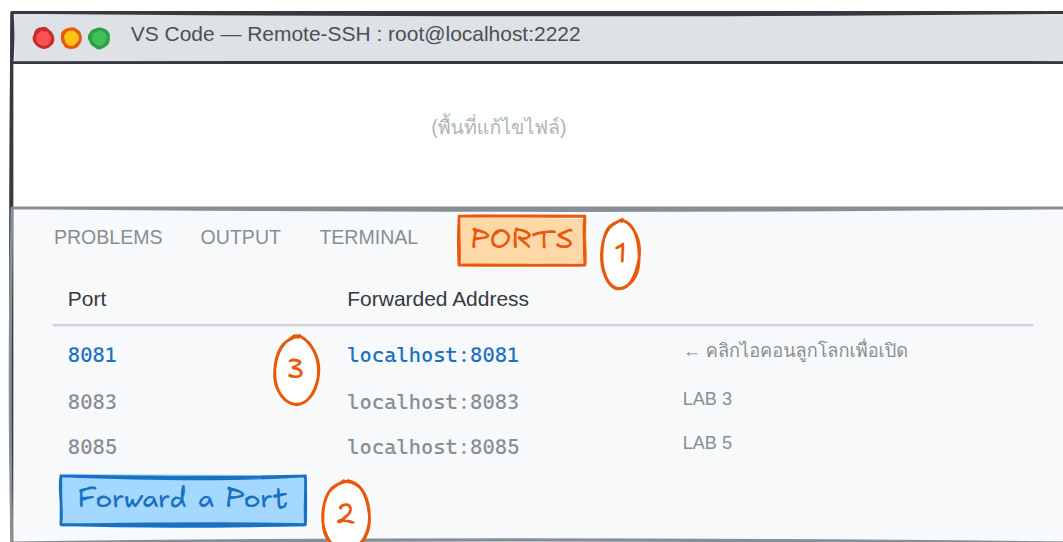
 คลิกเพื่อดูเฉลย

1) ไม่ — `requirements.txt` ไม่เปลี่ยน layer บนจึง CACHED · 2) แก้โค้ดตัวเดียว layer pip พังจาก cache — ติดตั้งใหม่ทุกรอบ

เปิดหน้าเว็บของแล็บ — Forward Port ใน VS Code

พอร์ตเปิดอยู่ "ข้างในเครื่องเรียน" — ให้ VS Code ทำ SSH tunnel ออกมา

Forward Port ใน VS Code — เปิดหน้าเว็บของแล็บในเบราว์เซอร์

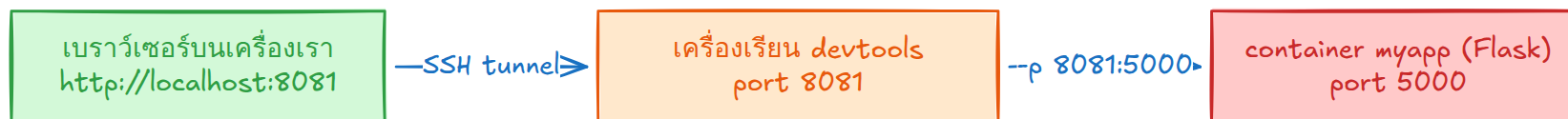


ขั้นตอน (ทำใน VS Code ที่ต่อ Remote-SSH อยู่)

- 1) เปิดแท็บ PORTS (แถวเดียวกับ TERMINAL)
- 2) กดปุ่ม Forward a Port
- 3) พิมพ์เลข port ของแล็บ เช่น 8081 แล้วกด Enter
- 4) เปิด `http://localhost:8081` ในเบราว์เซอร์

ทำไมต้อง forward? — port 8081 เปิดอยู่ใน "เครื่องเรียน devtools" ไม่ใช่บนเครื่องเราโดยตรง VS Code จะสร้าง SSH tunnel ให้อัตโนมัติ พอ forward แล้ว localhost:8081 บนเครื่องเรา จึงวิ่งเข้าถึงเว็บในแล็บได้

Port ของแต่ละแล็บ : LAB 1 → 8081 · LAB 3 → 8083 · LAB 5 → 8085, 8086



เส้นทางของ request : เบราว์เซอร์ → SSH tunnel ของ VS Code → port 8081 ในเครื่องเรียน → myapp ในกล่อง (port 5000)

หลักฐานจริง : v1 → v2 โดยไม่แก้ Dockerfile



myapp:1.0 — เวอร์ชัน 1.0 จาก ENV



myapp:2.0 + -e APP_VERSION=2.0

ป้าย "2.0" **ไม่ได้อยู่ในไฟล์ไหนเลย** — Dockerfile ยังเขียน 1.0 · build ครั้งเดียว รันเป็นเครื่องพัฒนา/ทดสอบ/ใช้งานจริง (dev/staging/prod)



| สรุป Section 1

- ▶ Dockerfile = สูตรข้อความ → `docker build` → image → `docker run` → container
- ▶ layer ซ้อนกัน — cache ตัดสินทีละชั้น ของเปลี่ยนแปลงไว้อย่าง
- ▶ EXPOSE คือป้าย · ตัวเปิดพอร์ตจริงคือ -p
- ▶ -e ตอนรัน ขณะที่ ENV ตอน build

CMD มีความลับ — พิมพ์อะไรต่อท้าย `docker run` มันหายทั้งบรรทัด! ไป Section 2



02

CMD vs ENTRYPOINT

คำถามสัมภาษณ์งานยอดฮิต — วันนี้ตอบด้วยการรันจริง ไม่ใช่การท่องจำ



ปัญหา : คำสั่งเดียวกัน ผลตรงข้าม

ลองพิมพ์ date ต่อท้าย docker run กับ image สองตัว

```
$ docker run --rm image-A date
Wed Aug 12 11:50:49 UTC 2026
# ได้วันเวลา - date ถูกจริง
```

```
$ docker run --rm image-B date
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
| | | | _ _ | | | _ _ _ _ _ | | _ _ | | _ _
# ได้ตัวอักษร ASCII!? date ไม่ถูก
```

ทำไม? — ขึ้นกับว่า image ประกาศคำสั่งด้วย **CMD** หรือ **ENTRYPOINT** · section นี้พิสูจน์ให้เห็นครบทุกกรณี



นิยาม : CMD กับ ENTRYPOINT

CMD — คำเริ่มต้น

รันเมื่อผู้ใช้ไม่พิมพ์อะไรต่อท้าย
พิมพ์ต่อท้าย → แทนที่ CMD ที่ก๊อ

ENTRYPOINT — โปรแกรมหลัก

รันเสมอ ไม่หายไปไหน
สิ่งที่พิมพ์ต่อท้าย → เป็น **argument** ต่อท้าย

สูตรมาตรฐาน : ใช้ **คู่กัน** — ENTRYPOINT ล็อกโปรแกรม · CMD เป็น argument ตั้งต้นที่เปลี่ยนได้



กายจากนิยาม

(figlet = โปรแกรมวาดข้อความเป็น ASCII ตัวใหญ่)

- 1 CMD ["figlet", "Hi"] – รัน ... date ได้อะไร?
- 2 ENTRYPOINT ["figlet"] – รัน ... date ได้อะไร?

 **คลิกเพื่อดูเลย**

1) วันเวลาจริง – date แทน CMD ทั้งก่อน · **2)** ASCII คำว่า date – ถูกต่อท้ายเป็น figlet date



เครื่องมือทดลอง : Dockerfile 3 แบบ + figlet

figlet = โปรแกรมวาดข้อความเป็น ASCII ตัวใหญ่ — เห็นผลต่างชัดในบรรทัดเดียว

```
# Dockerfile-CMD
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y figlet ...
```

CMD ["figlet", "Hello CMD"]

ค่าเริ่มต้น เปลี่ยนได้ทั้งก่อน

```
# Dockerfile-ENTRYPOINT
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y figlet ...
```

ENTRYPOINT ["figlet", "Hello"]

โปรแกรมหลัก args ถูกต่อท้าย

```
# Dockerfile-BOTH
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y figlet ...
```

ENTRYPOINT ["figlet"]
CMD ["Hello Docker"]

สูตรมาตรฐานของ image ดี ๆ

ทั้งสามไฟล์ FROM+RUN เหมือนกันเป๊ะ — จงใจ! เดี่ยวเห็น cache ช้ำมไฟล์



ไออะแกรม : กติกาทั้งหมดในตารางเดียว

อ่านทีละแถว : Dockerfile แบบนี้ + รันแบบนี้ → คำสั่งจริงคืออะไร

CMD vs ENTRYPOINT — ใครถูกแทนที่ ใครถูกต่อท้าย (จาก LAB 2)

Dockerfile กำหนดไว้ว่า...	docker run image (ไม่ใส่ argument)	docker run image date (ใส่ argument — เหมือนในแล็บ)
CMD ["figlet","Hello CMD"]	figlet "Hello CMD"	date ← แทนที่ก่อน! figlet หาย (รันคำสั่ง date ได้วันเวลาจริง)
ENTRYPOINT ["figlet","Hello"]	figlet "Hello"	figlet "Hello" date ← ต่อท้าย (วาดคำว่า date ไม่ใช่รันคำสั่ง!)
ENTRYPOINT ["figlet"] CMD ["Hello Docker"]	figlet "Hello Docker" (CMD = default args)	figlet date (args ใหม่แทนเฉพาะ CMD)

- CMD = ค่าเริ่มต้นที่เปลี่ยนง่าย — argument ท้าย docker run แทนที่ก่อน (ต้องเป็นคำสั่งที่มีจริง เช่น date)
- ENTRYPOINT = โปรแกรมหลักที่รันเสมอ — argument ถูก "ต่อท้าย" (อยากเปลี่ยนจริงต้องใช้ --entrypoint)
- สูตรมาตรฐาน : ENTRYPOINT คือโปรแกรม + CMD คือ default argument



```
$ docker run --rm cmd-example date
Wed Aug 12 11:50:49 UTC 2026
# CMD ถูกแทนที่ทั้งก้อน → date รั้นจริง
# figlet หายไปเลย
```

[illegible]

image-A ในสไลด์ปัญหา = `cmd-example · image-B = entrypt-example` — ปริศนาคลี่คลาย



| เครื่องมือ debug คู่ของ CMD/ENTRYPOINT

บังคับเปลี่ยน (--entrypoint) + ตรวจสอบจริง (inspect)

--entrypoint : บังคับเปลี่ยนโปรแกรมหลัก

```
docker run --rm --entrypoint date entrypoint-example
```

ต้องวาง**ก่อน**ชื่อ image · ใช้บ่อยตอน debug เช่น --entrypoint bash

docker inspect : อ่านค่าที่ฝังใน image

```
docker inspect --format '{{json .Config.Entrypoint}} {{json .Config.Cmd}}' <image>
```

เห็นค่าจริงโดยไม่ต้องเดา — null = ไม่ได้กำหนด



Error ที่เจอบ่อยใน section นี้

อาการ	สาเหตุ + ทางแก้
failed to read dockerfile: open Dockerfile: no such file	ไฟล์ชื่อไม่ตรง — ใส่ <code>-f Dockerfile-CMD</code>
พิมพ์ <code>date</code> ได้ ASCII แทนวันเวลา	ENTRYPOINT — ถูกแล้ว! อย่ากรั่นจริงใช้ <code>--entrypoint</code>
จาก <code>Exited</code> กองใน <code>docker ps -a</code>	ลบ <code>--rm</code> — เก็บด้วย <code>docker container prune -f</code>



สูตร ENTRYPOINT + CMD

- 1 ENTRYPOINT ["figlet"] + CMD ["Hello"] — รันเปล่า ๆ ได้อะไร?
- 2 รันแบบเดิมแต่ต่อท้ายด้วย "Bye" — อะไรถูกแทนที่?

 **คลิกเพื่อดูเฉลย**

1) figlet Hello — สองค่าต่อกัน · 2) เฉพาะ CMD → กลายเป็น figlet Bye ส่วน figlet ยังอยู่เสมอ



โพลาไรซ์ 002 LAB CMD vs ENTRYPOINT ~15 นาที มี verify.sh

- ▶ build 3 image — ตัวแรก ~12-30 วิ ที่เหลือ **CACHED ~1 วิ**
- ▶ รันครบ 6 กรณี (3 image × มี/ไม่มี args)
- ▶ วาดชื่อตัวเอง (อังกฤษ): ... figlet "YourName"
- ▶ --entrypoint + docker inspect ยืนยัน metadata

```
$ docker run --rm cmd-example figlet  
"Somchai"
```

```

/ _ | _ _ _ _ _ | | _ _ _ ( )
\ _ \ / _ \ | ' _ ` _ \ / _ | ' _ \ / _ ` | |
 _ ) | ( _ | | | | | | ( _ | | | | ( _ | | |
| _ _ / \ _ _ / | _ | | _ | _ \ _ _ | _ | | _ \ _ , | _ |

```



| สรุป Section 2

- ▶ args ทำ docker run → แทนที่ **CMD** ที่ก่อน · ต่อท้าย **ENTRYPOINT**
- ▶ ใช้คู่กัน : ENTRYPOINT = โปรแกรมคงที่ · CMD = argument ตั้งต้น
- ▶ --entrypoint คือทางหนีไฟ · docker inspect คือเครื่องตรวจ

image เราตอนนี้เก่งแล้ว — แต่ยังอยู่**แค่ในเครื่องเรา** · Section 3 : ส่งมันขึ้นคลาวด์



03

Docker Registry — ส่ง Image สู่มโลก

ได้เวลาส่งผลงานขึ้น Docker Hub ให้ใครก็ pull ไปรันได้



| ปัญหา : image ติดอยู่ในเครื่องเรา

- ▶ เพื่อนอยากลองเว็บเรา — ส่ง image ข้ามเครื่องยังไง?
ส่ง zip? ให้ build เอง? ล้วนพัง
- ▶ เซิร์ฟเวอร์จริงต้อง deploy — เอา image ไปไว้ไหน?
- ▶ `docker pull nginx` ดึงมาจากไหน?

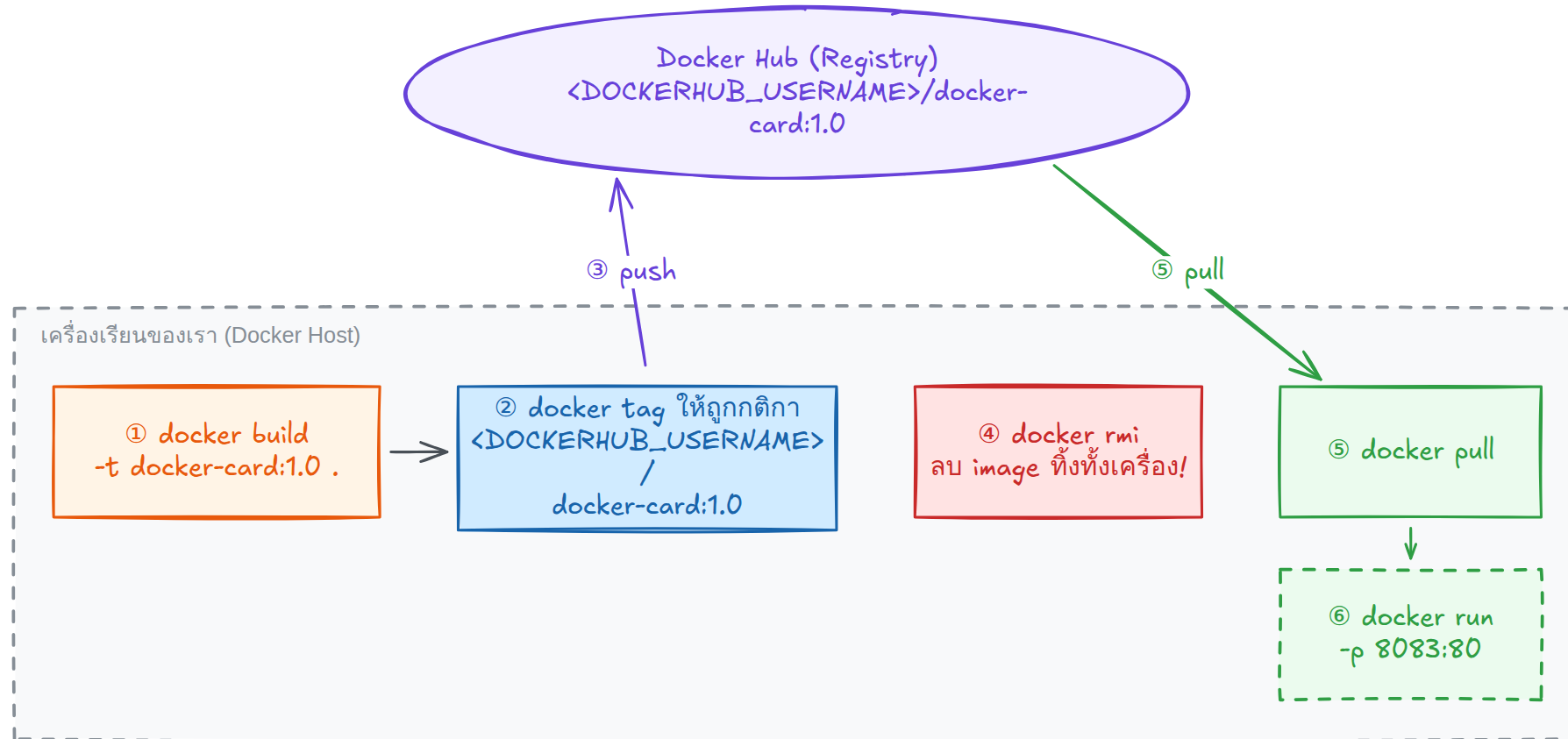
คำตอบเดียวกันทั้งสามข้อ : **registry** — "บ้านบนคลาวด์" ของ image



นิยาม : Registry = ที่เก็บและแจกจ่าย image

build ที่เครื่องเรา → push ขึ้น registry → เครื่องอื่น pull ไปรันได้ · Docker Hub คือบริการ registry สาธารณะที่นิยมใช้

เส้นทางของ LAB 3 : ส่งนามบัตรขึ้น Docker Hub แล้ว pull กลับมารัน



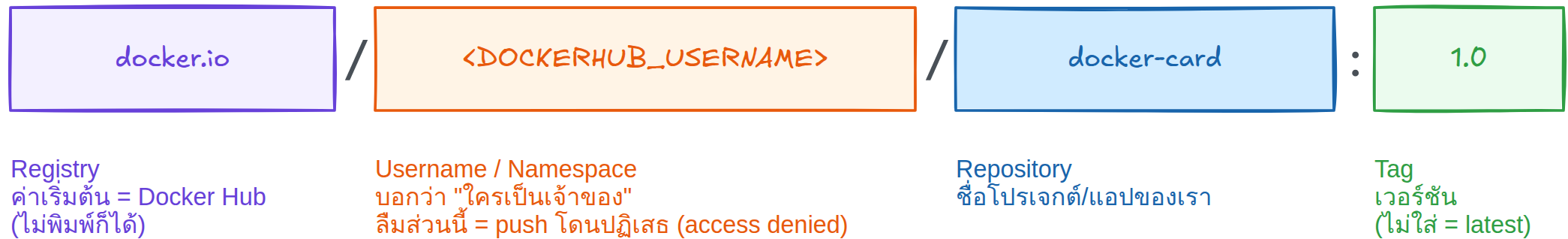
ลบทุกอย่างในเครื่องแล้ว image ยังอยู่บน Hub — ใครก็ pull ไปรันได้จากทุกเครื่องบนโลก 🌐 (นี่คือวิธีที่ image ทุกตัวใน LAB ถูกส่งมาถึงเรา)



| โดอะแกรม : กติกาชื่อ image — ชื่อผิด push ไม่ขึ้น

registry/username/repo:tag — LAB 3 จะพา push แบบผิดให้โดนปฏิเสธก่อน

กติกาการตั้งชื่อ Image — ถ้าชื่อผิด push ไม่ขึ้น!



✗ `docker push docker-card:1.0` → ไม่มี username → ถูกมองเป็น `docker.io/library/...` → denied

✓ `docker tag docker-card:1.0 <DOCKERHUB_USERNAME>/docker-card:1.0` แล้วค่อย push

tag คือ "ป้ายชื่อ" อีกใบของ image เดิม — IMAGE ID เดียวกัน ไม่ได้ก๊อปปี้ ไม่เปลี่ยนพื้นที่เพิ่ม



| อ่านชื่อ image ให้ออก

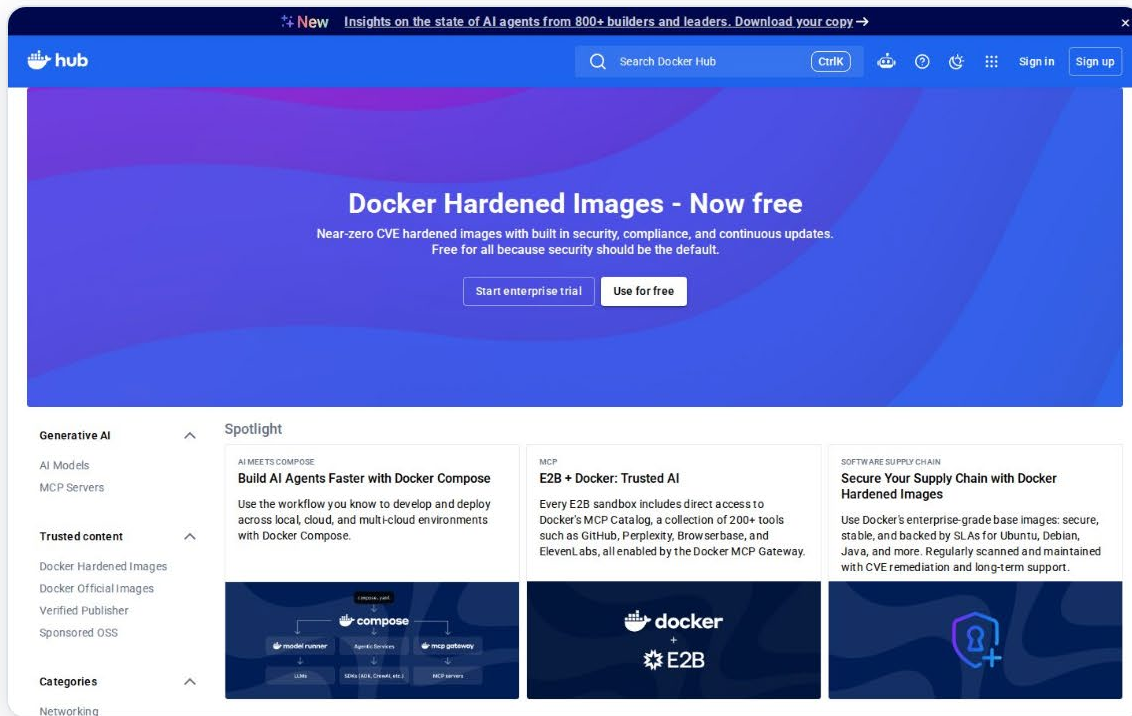
- 1 พิมพ์แค่ nginx — ชื่อเต็มคืออะไร?
- 2 docker push docker-card:1.0 สำเร็จไหม?

 คลิกเพื่อดูเฉลย

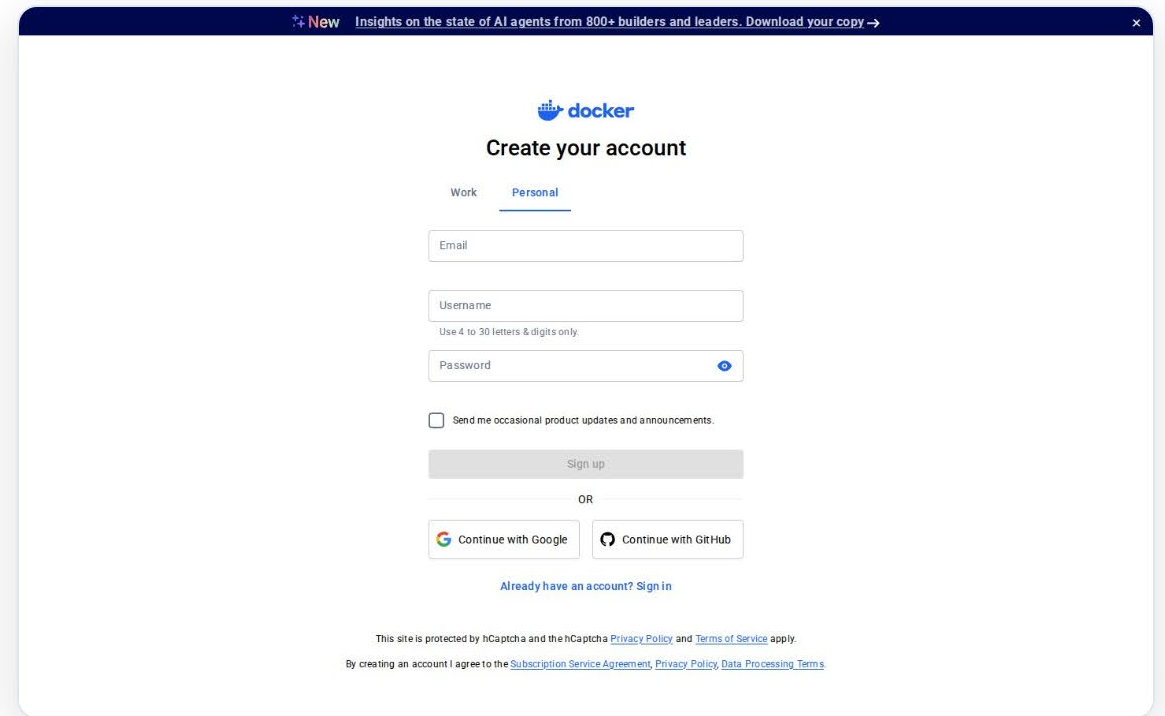
1) `docker.io/library/nginx:latest` — ค่า default ทั้ง registry/user/tag · **2)** ไม่ — ตกไป library/ พื้นที่ official ที่เราเขียนไม่ได้

หน้าจจริง ① : เข้า hub.docker.com → สมัคร / Sign in

สมัครฟรี — จำ **username** ให้แม่น เพราะมันจะกลายเป็นส่วนหนึ่งของชื่อ image เรา



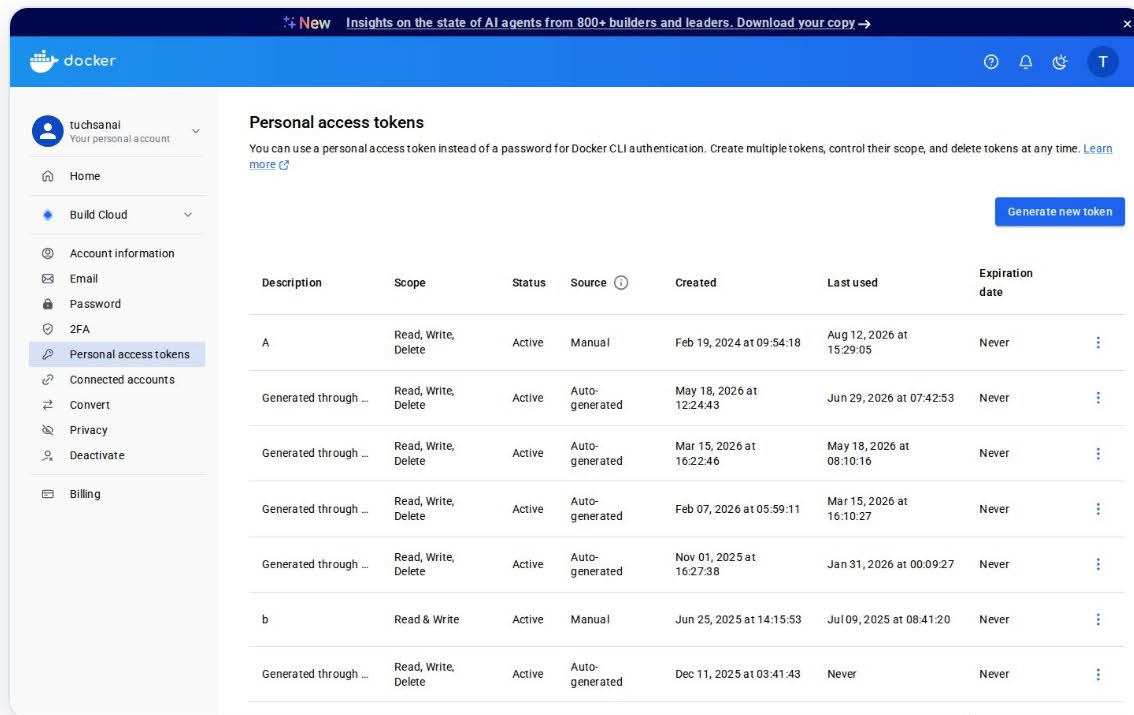
① hub.docker.com — ปุ่ม Sign up / Sign in มุมขวาบน



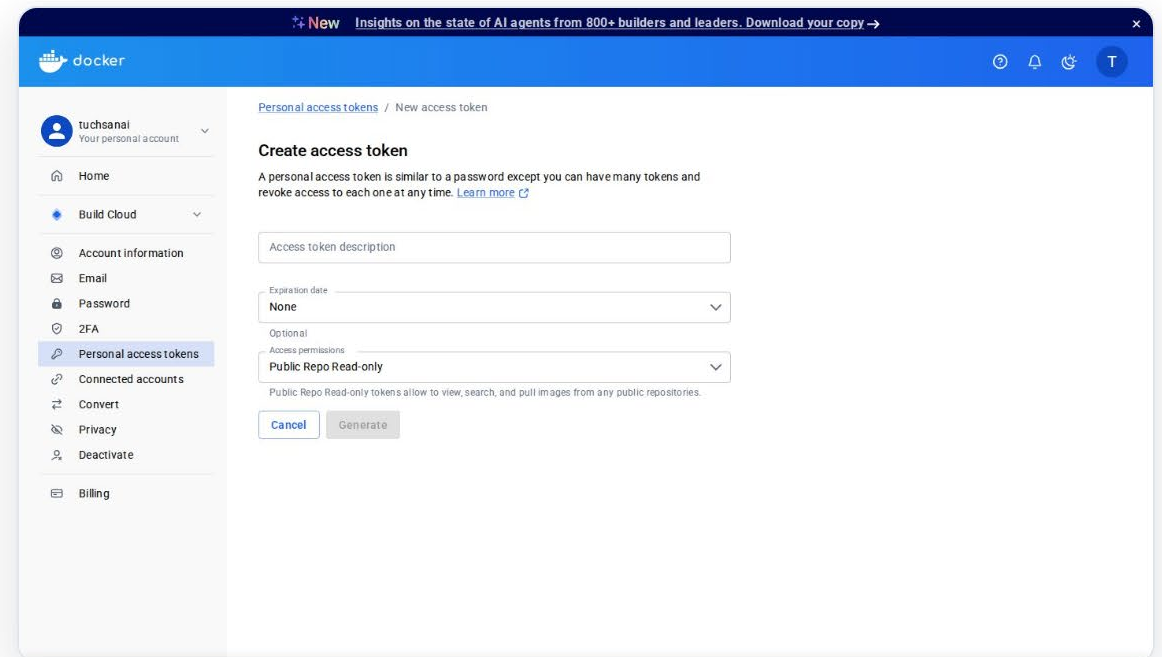
② ฟอรมสมัคร — อีเมล · username (4–30 ตัว) · รหัสผ่าน

หน้าจจริง ② : สร้าง Access Token ที่ละคลิก

รูปโปรไฟล์ → Account Settings → Personal access tokens → Generate new token



③ หน้า Personal access tokens (หมวด Security)



④ ตั้งชื่อ token · สิทธิ์ **Read & Write** · ไขว้ครั้งเดียว — คัดลอกเก็บทันที!



คำสั่งจริง : login ด้วย Access Token

```
$ docker login -u <DOCKERHUB_USERNAME>
```

Password: ← วาง Access Token (ไม่ใช่รหัสผ่านจริง!)

Login Succeeded

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'

ทำไมต้อง token

จำกัดสิทธิ์ได้ · ตั้งหมดอายุได้ · หลุดแล้วเพิกถอนได้ทันที

ข้อควรระวัง

ถ้าไม่มี credential helper token อาจอยู่ใน config.json · ใช้เสร็จให้
docker logout · ห้าม commit token



คำสั่งจริง : tag ให้ถูกกติกา แล้ว push

docker tag = ติดป้ายชื่อเพิ่ม ไม่ใช้การ copy

```
$ docker tag docker-card:1.0 <DOCKERHUB_USERNAME>/docker-card:1.0
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE
docker-card:1.0	76f06bd3bded	73.6MB	21MB
<DOCKERHUB_USERNAME>/docker-card:1.0	76f06bd3bded	73.6MB	21MB

↑ สองชื่อ ID เดียวกันปะ๊ะ — ดิสก์ถูกใช้ก่อนเดียว

```
$ docker push <DOCKERHUB_USERNAME>/docker-card:1.0
c016feea144c: Pushed
... (ทุก layer ขึ้น Pushed) ...
1.0: digest: sha256:76f06bd3bded... size: 856
```

IMAGE ID เดียวกัน = สอง tag ชี้ image ก้อนเดียวกัน · **digest** หลัง push = ลายนิ้วมือของสิ่งที่ registry เก็บ



Error ที่ทุกคนต้องเจอ : push โดนปฏิเสธ

LAB 3 จงใจให้ push ผิดก่อน — อ่าน error ให้จบแล้วจะเข้าใจกติกาชื่อตลอดไป

```
$ docker push docker-card:1.0
The push refers to repository [docker.io/library/docker-card]
c016feal144c: Waiting
push access denied, repository does not exist or may require authorization:
server message: insufficient_scope: authorization failed
```

สาเหตุ

ไม่มี username นำหน้า → ตกไป library/ ของ official — เอา
นามบัตรไปแปะบ้านคนอื่น

ทางแก้ (ครบทั้งคู่)

① docker login · ② docker tag ให้ขึ้นต้นด้วย username เรา



| tag กับ push

- 1 docker tag image 5GB — ช้าไหม? เปลืองดิสก์ไหม?
- 2 login แล้ว push ยังโดน denied — เพราะอะไร?

 **คลิกเพื่อดูเฉลย**

1) เสียเวลานานที ไม่เปลือง — tag คือป้ายชี้ก้อนเดิม · **2)** ชื่อไม่มี username นำหน้า หรือสะกดไม่ตรงกับบัญชีที่ login



Registry ฉลาดเรื่อง layer — push ซ้ำไม่เปลืองเน็ต

ผลจริงตอน push เวอร์ชัน 2.0 (แก้ html ไฟล์เดียว)

```
$ docker push <DOCKERHUB_USERNAME>/docker-card:2.0
197eb75867ef: Layer already exists
b464cfd2a63: Layer already exists
... (layer ของ nginx ทั้ง 8 ชั้น ขึ้น already exists) ...
82c6cbb690ec: Pushed ← อัปเดตจริงเฉพาะชั้นที่เปลี่ยน (ไม่กี่ kB)
2.0: digest: sha256:109ec430... size: 856
```

registry เก็บ layer ตามลายนิ้วมือ (digest) — รับเฉพาะชั้นใหม่ · เหตุผลเดียวกับที่ pull image ที่มี base ร่วมกันขึ้น Already exists



Registry ไม่ได้มีแต่ Docker Hub

องค์กรอาจใช้ GHCR · GitLab Container Registry · AWS ECR — โครงสร้างคล้ายเดิม แต่ขั้นตอน login ต่างกันตามผู้ให้บริการ

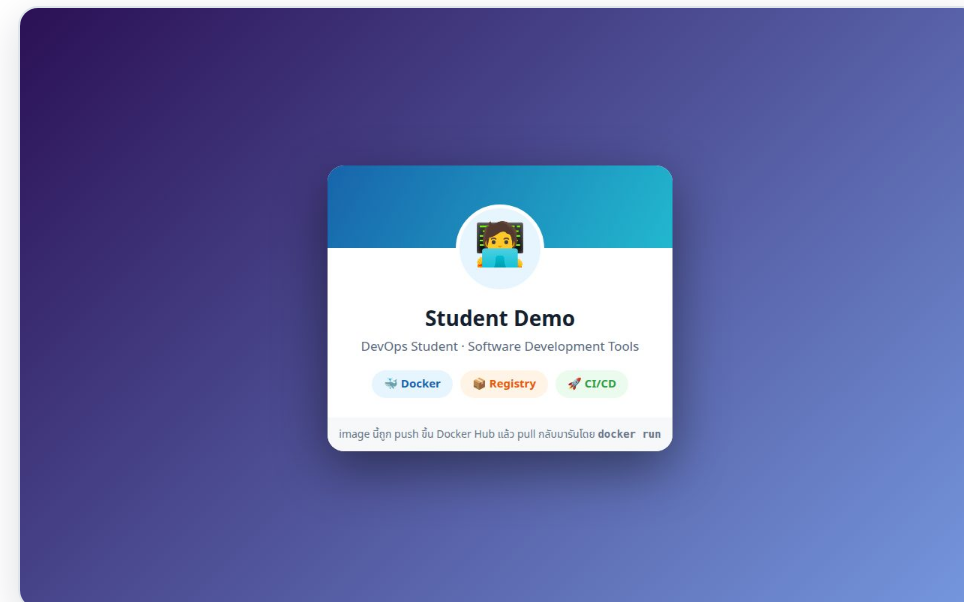
- ▶ เปลี่ยนแค่ส่วน registry ที่นำหน้าชื่อ image
- ▶ self-host เองได้ — registry ก็เป็นแค่ container!
ทำจริงใน LAB 3 ข้อ 6

```
# self-host registry ในเครื่องตัวเอง
$ docker run -d -p 5000:5000 --name registry
registry:2
$ docker tag docker-card:1.0 localhost:5000/docker-card:1.0
$ docker push localhost:5000/docker-card:1.0
$ curl -s localhost:5000/v2/_catalog
{"repositories":["docker-card"]}
```

ส่งนามบัตรออนไลน์ของคุณขึ้น Docker Hub

โพเดอร์ 003_LAB_Docker_Hub_Registry · ~20 นาที · มี verify.sh

- ▶ แก้ index.html ใส่ชื่อ แล้ว build docker-card:1.0
- ▶ **push ผิดก่อน** → access denied → เรียนจาก error
- ▶ token → login → tag → push
- ▶ self-host registry:2 push เข้าเครื่องตัวเอง
- ▶ ลบเกลี้ยง → pull กลับมารัน — ครบวงจร

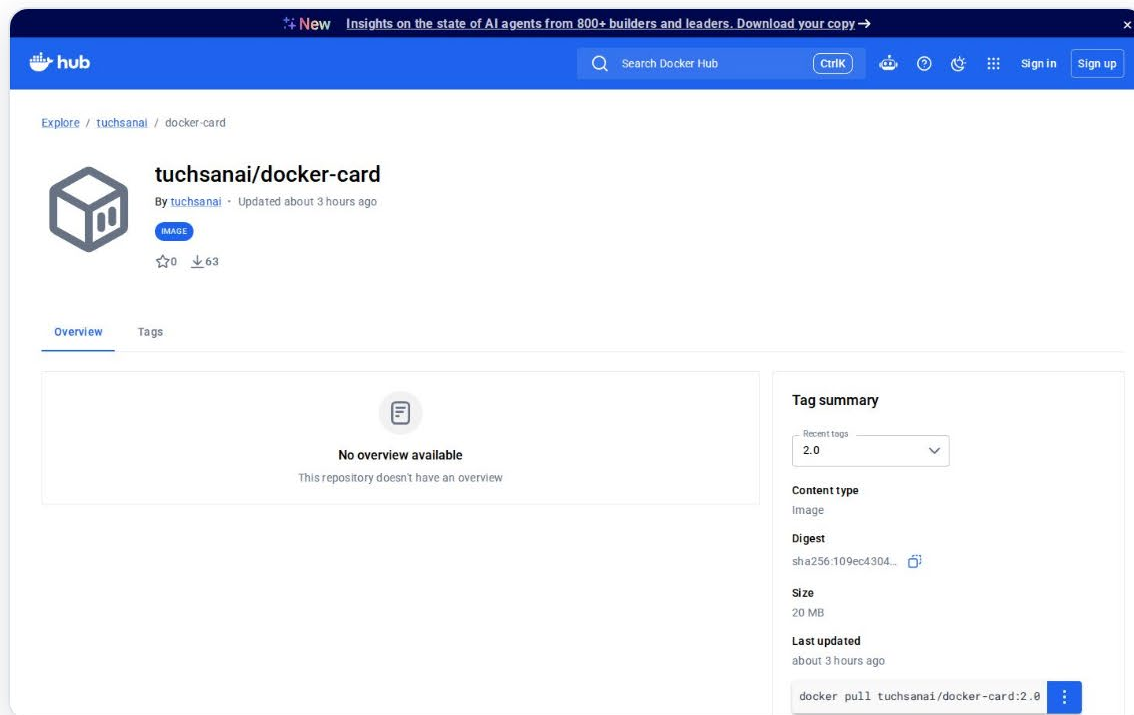


นามบัตรที่เพื่อน docker pull ไปเปิดได้จริง

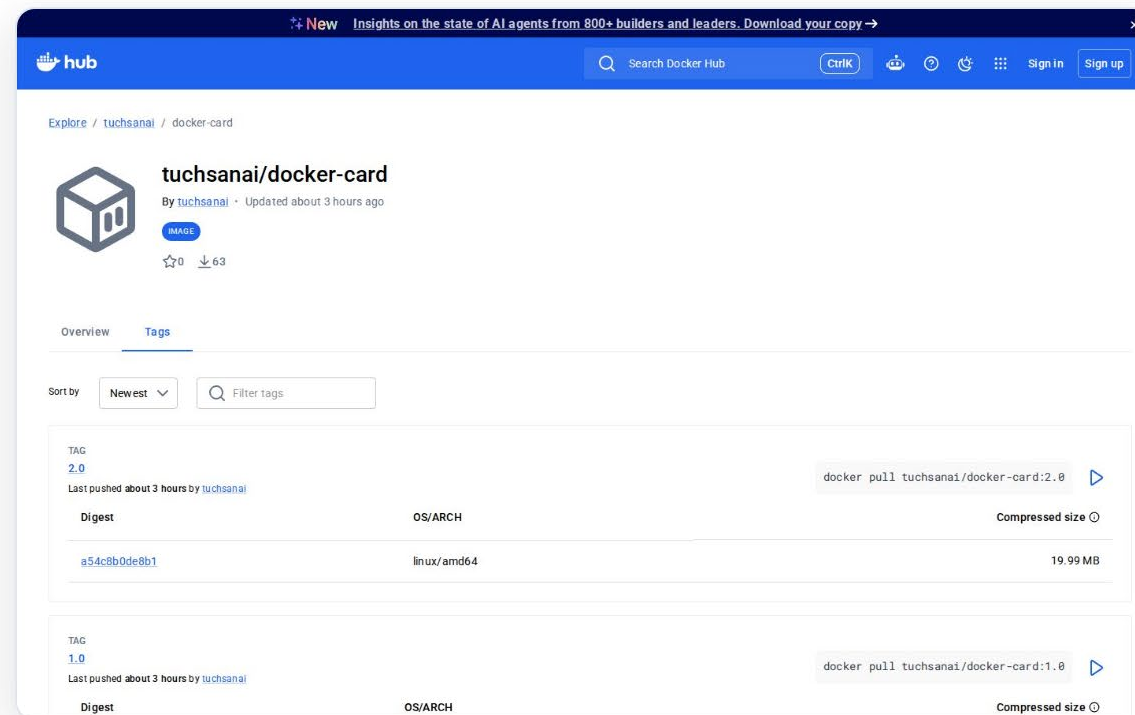


หลักฐานจริงบน Docker Hub หลัง push

repo docker-card ของจริงจากเว็บนี้ — capture สดจาก hub.docker.com



หน้า repo — digest ตรงกับค่าที่ Docker รายงานหลัง push



แท็บ Tags — เพื่อนคัดลอกคำสั่ง docker pull จากตรงนี้



| สรุป Section 3

- ▶ ชื่อ image คือที่อยู่ : registry/username/repo:tag
- ▶ login ด้วย **Access Token** — และ logout เมื่อเสร็จ
- ▶ tag = ป้าย ไม่ใช่ copy · registry รับเฉพาะ layer ใหม่
- ▶ เครื่องเรา = ที่ build/รัน · **บ้านจริงอยู่บน registry**

แต่ container ยัง "ตัวใครตัวมัน" — Section 4 : จับมาคุยกัน



ทวนก่อนลุยต่อ

- 1 ลบ image ทุกป้ายหลัง push — งานหายถาวรไหม?
- 2 ทำไม push เวอร์ชันใหม่ถึงเร็ว?

 **คลิกเพื่อดูเฉลย**

1) ไม่หาย ถ้า tag ยังอยู่บน registry และเรายังมีสิทธิ์ — pull กลับได้ · **2)** registry มี layer เดิมแล้ว รับเฉพาะชั้นใหม่ (Layer already exists)



04

Docker Network

แอปจริงไม่ได้มี container เดียว — web ต้องคุยกับ database ให้ได้



| ปัญหา : IP ของ container เปลี่ยนได้ทุกครั้งที่รัน

- ▶ รันใหม่ → แจก IP ใหม่ — เมื่อที่ .3 ตอนนี้อาจเป็น .5
- ▶ ฮาร์ดโค้ด IP ลง config = หายนะรอวันระเบิด
- ▶ แต่โค้ดจริงเขียน `Redis(host="redis")` — ใช้ชื่อแล้วทำงานได้!?
Redis = ฐานข้อมูลในหน่วยความจำ (ตัวเก็บคะแนนใน LAB 5)

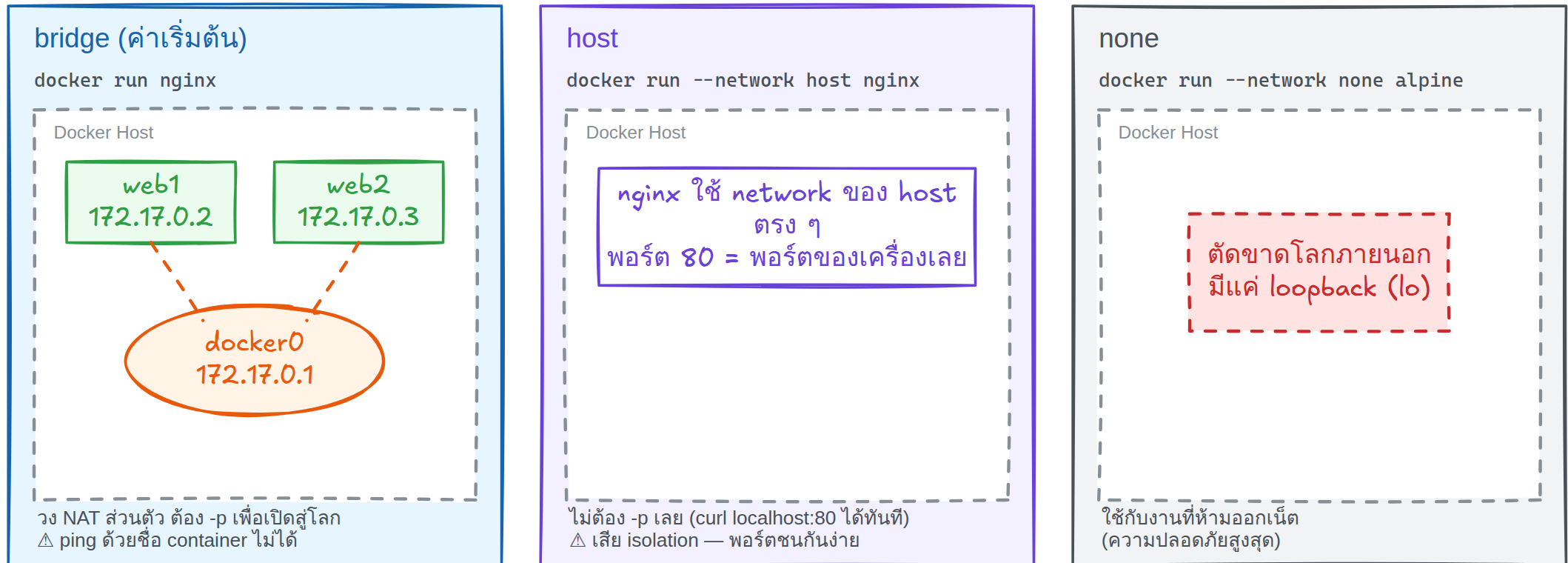
ใครแปลชื่อ redis เป็น IP? — เผลอด้วยการทดลองสองยก



ไออะแกรม : 3 network ที่ Docker แยกมา

network = วง LAN เสมือนให้ container คุยกัน · bridge = วง **NAT** (ตัวแปลง IP วงใน↔โลกนอก) — เลขวงแต่ละเครื่องต่างกันได้

3 โหมด Network ที่ Docker แยกมาให้ (docker network ls)



LAB 4 เริ่มที่ bridge ค่าเริ่มต้น แล้วไปสร้าง network ของเราเอง — ซึ่งแถม DNS ให้เรียกกันด้วยชื่อได้



คำสั่งจริง : บน default bridge — IP ได้ · ชื่อไม่ได้

ความล้มเหลวที่ "ถูกต้อง" — บทเรียนสำคัญที่สุดของเรื่องนี้

```
$ docker exec box1 ping -c 2 172.18.0.3
PING 172.18.0.3: 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=64
2 packets transmitted, 2 received,
0% packet loss
# ping ด้วย IP → ผ่านสบาย
```

```
$ docker exec box1 ping -c 2 box2
ping: bad address 'box2'
# ping ด้วยชื่อ → ไม่รู้จักเลย
# default bridge ไม่มีบริการแปลงชื่อ
```

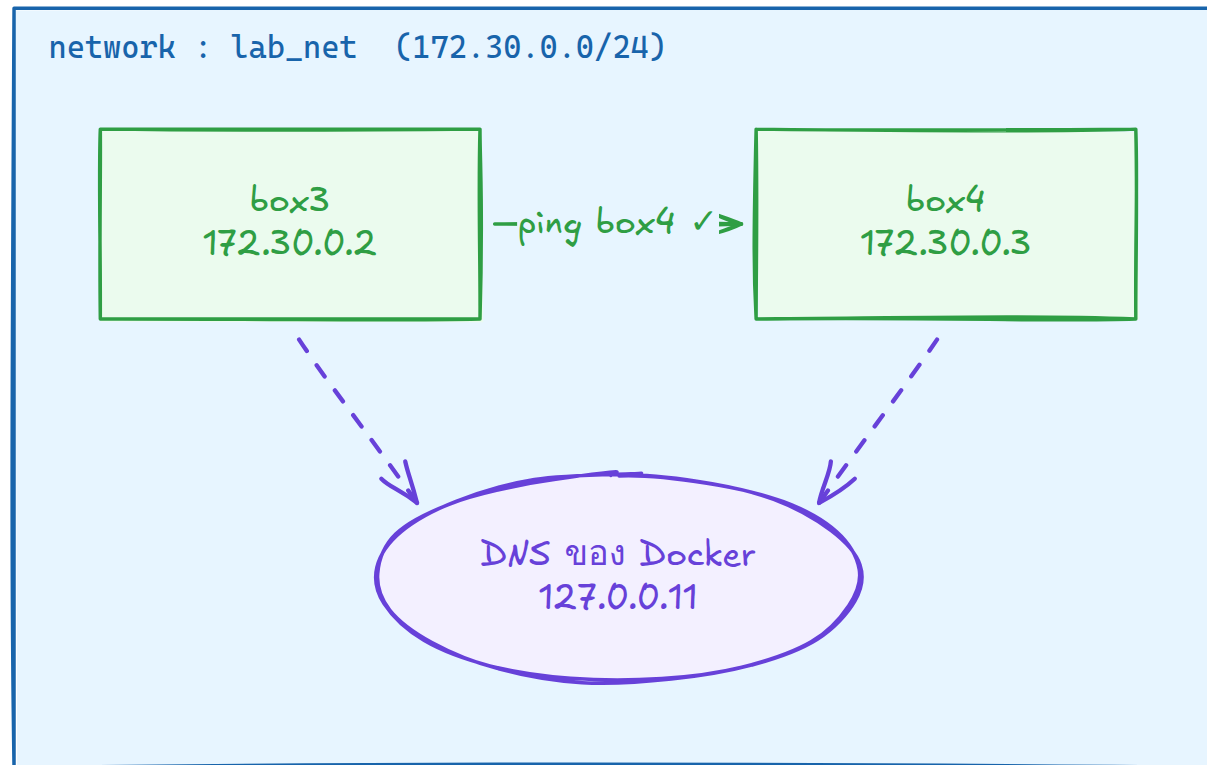
bad address = แปลชื่อไม่ได้ตั้งแต่ต้น (คนละอาการกับ "แปลได้แต่ต่อไม่ติด" ซึ่งขึ้น timeout)



User-defined Network — แอม Embedded DNS ฟรี

Embedded DNS = DNS ที่ Docker ผังไว้ที่ 127.0.0.11 ในทุกวงที่เราสร้างเอง — แปลชื่อ container → IP ปัจจุบันให้สด ๆ

Embedded DNS — ทำไม user-defined network เรียกกันด้วยชื่อได้



ตารางที่ DNS จำให้ (อัตโนมัติ)

ชื่อ container	→	IP
box3	→	172.30.0.2
box4	→	172.30.0.3
secret-server	→	172.30.0.4

ใน container : `cat /etc/resolv.conf`
→ `nameserver 127.0.0.11`

บน default bridge ไม่มีบริการนี้ (ping box2 → bad address) — นี่คือเหตุผลที่ได้จริงเขียน Redis(host="redis") แล้วใช้ได้เลยใน docker compose



| ชุดคำสั่ง network (LAB 4)

คำสั่ง	ทำหน้าที่
<code>network create --subnet 172.30.0.0/24 lab_net</code>	สร้างวง + subnet
<code>network inspect lab_net</code>	ดู subnet + สมาชิก
<code>run --network lab_net --ip 172.30.0.50</code>	เข้าวง + ล็อก IP
<code>network connect / disconnect lab_net spy</code>	เสียบ/ถอดสด ๆ
<code>network rm lab_net</code>	ลบวง

เริ่มสำรวจด้วย `docker network ls` · ทุกแถวขึ้นต้น `docker` · วงใครวงมัน มองไม่เห็นกัน



DNS กับการเสียบง

- 1 อยู่ network แบบไหน ถึงเรียกเพื่อนด้วยชื่อได้?
- 2 network connect ตัวที่รันอยู่ ต้อง restart ไหม?
- 3 ไฟล์ไหนบอกว่าใครแปลงชื่อให้?

 **คลิกเพื่อดูเฉลย**

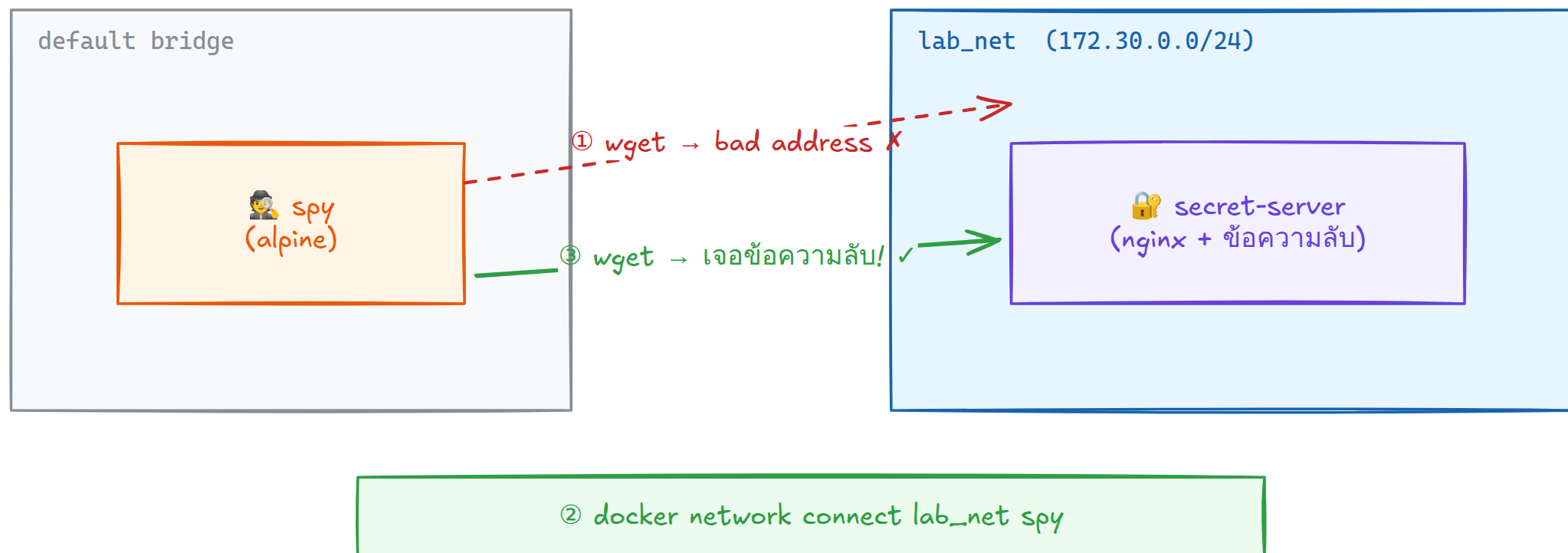
1) วงที่สร้างเอง — default bridge ไม่มี DNS · **2)** ไม่ — เสียบสด ๆ ได้ · **3)** `/etc/resolv.conf` → `127.0.0.11`



โตะแถม : การกักขังข้อมูลลับ

server ลับไม่มี -p — โลกภายนอกมองไม่เห็น เห็นเฉพาะสมาชิก lab_net

การกักขังข้อมูลลับ  — docker network connect เลียบสาย LAN ให้ container ที่กำลังรันอยู่



คนละ network = มองไม่เห็นกัน (isolation) · connect แล้ว container มี 2 ขา อยู่ทั้งสองวงพร้อมกัน — ถอดออกได้ด้วย `docker network disconnect`



โหมดพิเศษ : host และ none

สองขั้วของ isolation (การแยกวงไม่ให้เห็นกัน) — ผลจริงจากเครื่องเรียน

```
$ docker run -d --network host \
  --name hostnginx nginx:alpine
$ curl -s localhost:80 | grep title
<title>Welcome to nginx!</title>
# ไม่มี -p สักตัว แต่ curl เจอทันที!
```

host — ถอดกำแพงทิ้ง

ใช้ network เครื่องตรง ๆ · เร็ว ไม่มี NAT · แลกกับพอร์ตชนง่าย เสีย isolation

```
$ docker run --rm --network none \
  alpine ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> ...
    inet 127.0.0.1/8 scope host lo
# มีแค่ loopback — ออกเน็ตไม่ได้เลย
```

none — ไม่มี network ภายนอก

เหมาะกับงานที่ไม่ต้องใช้ network · ไม่ได้แทนมาตรการความปลอดภัยอื่น



Error ที่เจอบ่อยใน section นี้

อาการ	สาเหตุ + ทางแก้
<code>ping: bad address 'box2'</code> บน default bridge	ไม่ใช่ความผิดพลาด — วงนี้ไม่มี DNS
<code>user specified IP address is supported on user defined networks only</code>	<code>--ip</code> ใช้ได้เฉพาะวงที่มี <code>--subnet</code>
<code>network rm</code> พ้อง <code>has active endpoints</code>	ยังมีตัวเสียอยู่ — ถอดสมาชิกก่อน

Network Detective — การทึ่งกู้ข้อความลับ

โพเดอร์ 004_LAB_Docker_Network_DNS · ~20 นาที · CLI ล้วน · มี verify.sh

- ▶ default bridge : IP ผ่าน · ชื่อ → bad address
- ▶ lab_net → ping ชื่อสำเร็จ · resolv.conf เจอ DNS ลับ
- ▶ ล็อก IP ด้วย --ip 172.30.0.50
- ▶ spy : connect กู้ข้อความลับ → disconnect แล้วต้องมีดบอดอีกครั้ง
- ▶ ทัวร์ --network host และ none

```
$ docker exec spy wget -qO- http://secret-server
```

```
<h1>ยินดีด้วย! คุณกู้ข้อความลับสำเร็จ 🎉</h1>
```

```
$ docker inspect -f '{{json .NetworkSettings.Networks}}' spy
"bridge": { "IPAddress": "172.18.0.4",
"DNSNames": null }
"lab_net": { "IPAddress": "172.30.0.5",
"DNSNames": ["spy",...] }
# spy มี 2 ขา — มีชื่อใน DNS เฉพาะวง lab_net
```



| สรุป Section 4

- ▶ default bridge : IP ได้ ชื่อไม่ได้ — และ IP ไม่เที่ยง
- ▶ วงสร้างเอง : **Embedded DNS** — เบื้องหลัง Redis (host="redis")
- ▶ connect/disconnect ปรับผังสด ๆ ไม่ต้อง restart
- ▶ host เร็วแต่เปลือง · none ตัดขาดโลก

แต่เราพิมพ์เองทีละคำสั่ง — พอมี 5 service ละ? · Section 5 : ทั้งหมดในไฟล์เดียว



host / none

- 1 งานข้อมูลลับ ห้ามออกเน็ต — ใช้โหมดไหน?
- 2 เว็บที่อยากให้เร็วสุด ไม่ต้อง NAT ยอมเสีย isolation — โหมดไหน?

 คลิกเพื่อดูเฉลย

1) `--network none` — เหลือแค่ loopback · 2) `--network host` — ใช้ network เครื่องตรง ๆ ระวังพอร์ตชน



05

Docker Compose

ประกาศทั้งระบบในไฟล์เดียว แล้วเปิดด้วยคำสั่งเดียว



ปัญหา : เปิดระบบด้วยมือ = พิมพ์เยอะ พลาดง่าย

ยุคก่อนเชื่อม container ด้วย `--link` (legacy – ยังเจอในเอกสารเก่า)

```
$ docker run -d --name=redis redis
$ docker run -d --name=db postgres:9.4
$ docker run -d --name=vote -p 5000:80 --link redis:redis voting-app
$ docker run -d --name=result -p 5001:80 --link db:db result-app
$ docker run -d --name=worker --link db:db --link redis:redis worker
```

แนวทางใหม่ใช้ user-defined network หรือ **Compose** · เจอ `links` : ในระบบเก่าให้ทำความเข้าใจก่อน แล้วค่อยย้าย—อย่าลบทันที



นิยาม : Docker Compose

ประกาศ "ทั้งระบบ" ในไฟล์ **YAML** (ไฟล์ข้อความรูปแบบ key: value) เดียว แล้วเปิด/ปิดด้วย `docker compose up / down`

ทำไมต้องมี Docker Compose — จากหลายคำสั่ง เหลือไฟล์เดียว + คำสั่งเดียว

แบบเดิม : `docker run` ทีละตัว 🥲

```
docker network create back-tier ...
```

```
docker volume create vote-data
```

```
docker run -d --name redis --network ... -v ...
```

```
docker run -d --name vote -p 8085:5000 ...
```

```
docker run -d --name result -p 8086:5000 ...
```

พิมพ์เยอะ · ผิดง่าย · ต้องจำลำดับ · แชร์ให้เพื่อนทำตามยาก

แบบ `compose` : ประกาศทั้งระบบในไฟล์เดียว 😎

```
docker-compose.yml
```

```
services:
  vote:    build: ./vote
  result:  build: ./result
  redis:   image: redis:7-alpine
volumes:
  vote-data:
networks:
  front-tier:
  back-tier:
```

`docker
compose up`

อ่านง่าย · เก็บใน git ได้ · ทีมรันซ้ำได้เหมือนกันทุกเครื่อง

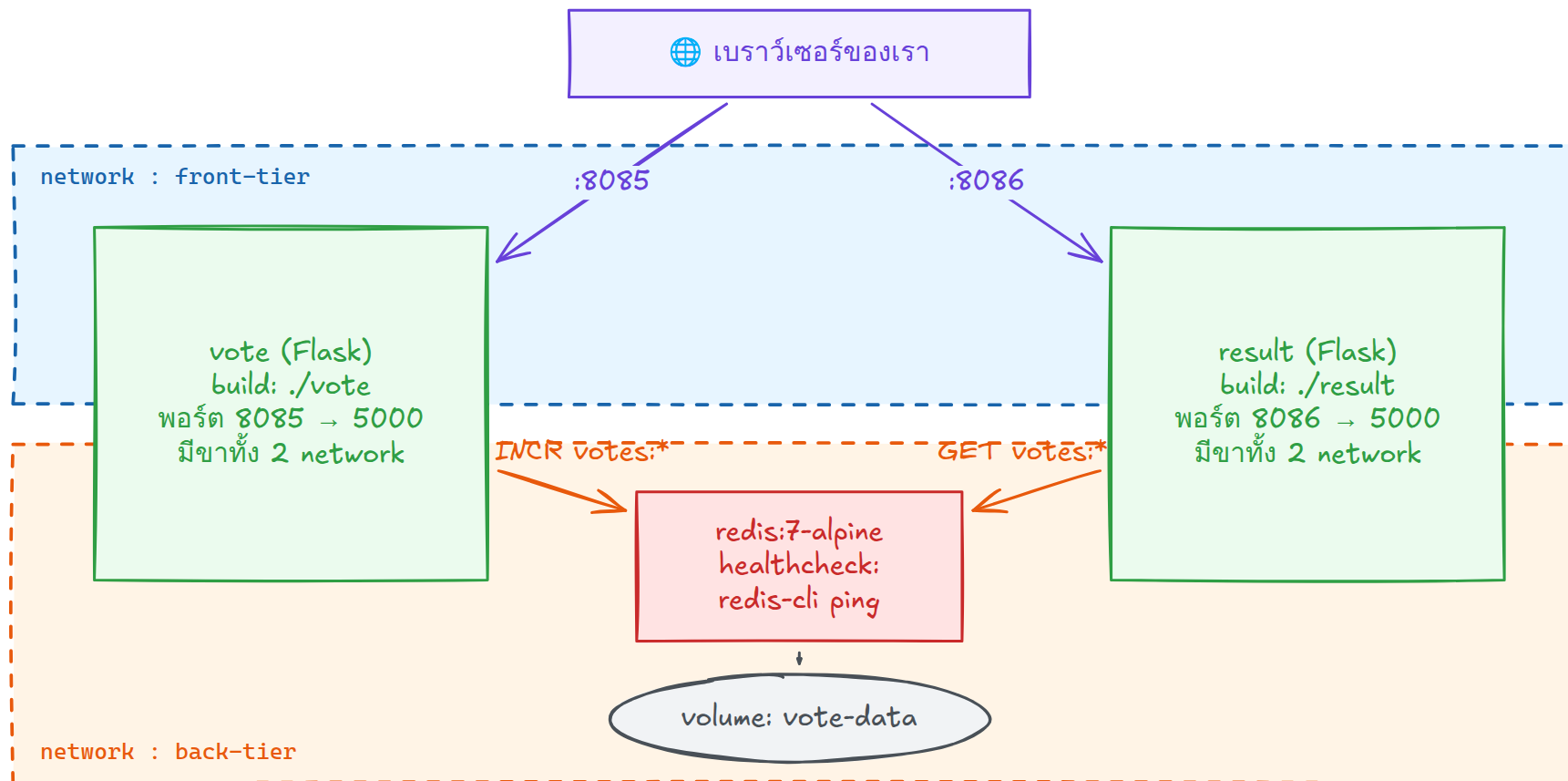
Infrastructure as Code ฉบับเริ่มต้น — โครงสร้างระบบกลายเป็นไฟล์ที่ review และ version control ได้



ไออะแกรม : สถาปัตยกรรม LAB 5 — Cats vs Dogs

ย่อส่วนจาก example-voting-app ตัวจริงของ Docker (ตัด worker/Postgres ให้ build ไว้)

สถาปัตยกรรม LAB 5 : Cats vs Dogs — 3 services · 2 networks · 1 volume



ทุก service เรียกกันด้วย "ชื่อ service" ผ่าน DNS ของ compose network · redis ไม่มีขาใน front-tier — เปราว์เซอร์เข้าตรง ๆ ไม่ได้ (ปลอดภัยขึ้น)



อ่าน docker-compose.yml ① – ประกาศ service

ไฟล์จริงจาก LAB 5 (ตัดมาเฉพาะส่วน vote/redis)

```
services:
  vote:
    build: ./vote           # build จาก Dockerfile เรา
    ports: ["8085:5000"]    # คู่ -p เดิม ในรูป YAML
    environment:
      REDIS_HOST: redis     # ใช้ host ด้วย "ชื่อ service"

  redis:
    image: redis:7-alpine  # ใช้ image สำเร็จรูป
```

services	1 service = 1 อนุภาคน
build / image	โค้ดเรา → build · สำเร็จรูป → image
ports	-p เดิม — redis ไม่มี = โลกนอกเข้าไม่ถึง
environment	-e เดิม — ชื่อ service ใช้ได้เพราะ DNS

compose ยุคใหม่**ไม่มี version:** — เจอในตัวอย่างเก่า ลบได้



| อ่าน docker-compose.yml ② – เจื่อนไขและทรัพยากร

```
vote:
  depends_on:
    redis:
      condition: service_healthy
  redis:
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
  volumes: [vote-data:/data]
  networks: [back-tier]
```

```
volumes:
  vote-data:
networks:
  front-tier:
  back-tier:
```

รอให้พร้อม

healthcheck ทาม "พร้อมไหม" ทุก 5 วิ ด้วย redis-cli ping

depends_on รอจน redis **healthy** — ไม่ใช่แค่ start

ทรัพยากรนอก container

volumes ข้อมูล redis อยู่นอกตัว — รอดการลบ

networks แยกโซน front/back — compose สร้างเอง



| อ่านไฟล์ compose

- 1 service ไหนใช้ build: ไหนใช้ image:?
- 2 service_healthy ต่างจาก depends_on สั้น ๆ ยังไง?
- 3 ทำไม redis จงใจไม่ประกาศ ports :?

คลิกเพื่อดูเฉลย

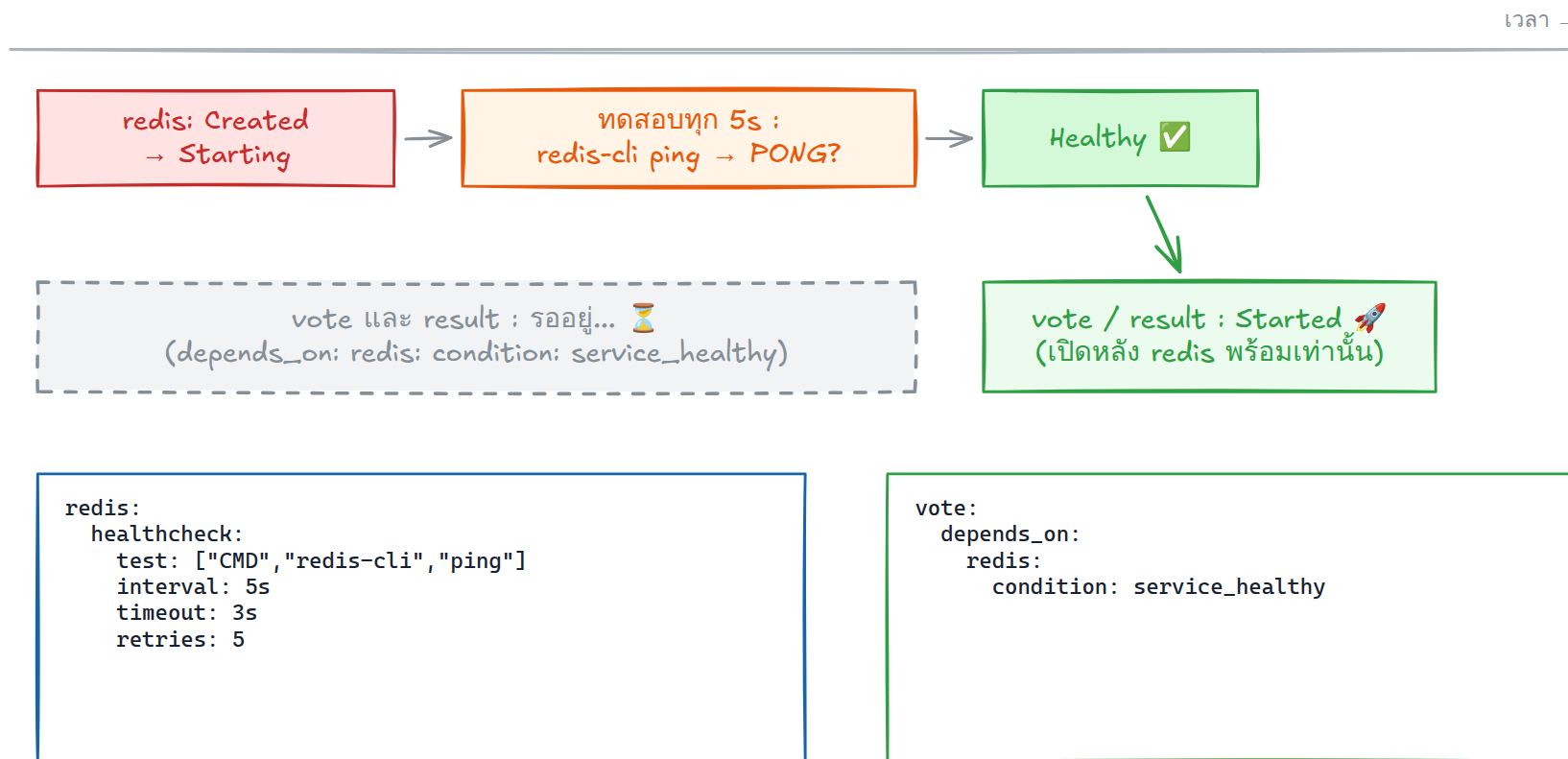
1) โค้ดเรา → build · สำเร็จรูป → image · 2) สั้นรอแค่ start — service_healthy รอ healthcheck ผ่าน · 3) ให้เห็นเฉพาะใน back-tier — โลกนอกรยังเข้า database ไม่ได้



ไอ้โง่แถม : healthcheck + depends_on

แก้ bug คลาสสิก "เว็บเปิดก่อน database พร้อม" → Internal Server Error

healthcheck + depends_on — เปิดเว็บเมื่อฐานข้อมูล "พร้อมจริง" เท่านั้น



ถ้าใช้ depends_on ธรรมดา (ไม่มี condition) จะรอแค่ "เริ่ม start" — เว็บอาจเปิดก่อน DB พร้อม แล้วพังด้วยอาการ Internal Server Error คลาสสิก



คำสั่งจริง : เปิดทั้งระบบในคำสั่งเดียว

```
$ docker compose up -d --build
Network ..._back-tier Created Volume ..._vote-data Created
Container ...-redis-1 Started
Container ...-redis-1 Waiting
Container ...-redis-1 Healthy ← รอกจนผ่าน healthcheck
Container ...-vote-1 Started ← เว็บค่อยเปิดตามเงื่อนไข
Container ...-result-1 Started

$ docker compose ps
NAME                SERVICE STATUS PORTS
...-redis-1         redis Up 17 seconds (healthy) 6379/tcp
...-result-1        result Up 12 seconds 0.0.0.0:8086->5000/tcp
...-vote-1          vote Up 12 seconds 0.0.0.0:8085->5000/tcp
```

compose ตั้งชื่อให้เอง : container = <โปรเจกต์>-<service>-<ลำดับ> · network/volume = <โปรเจกต์>_<ชื่อ> (โปรเจกต์ = ชื่อโวลูม)



| ชุดคำสั่ง compose ประจำวัน

คำสั่ง	ทำหน้าที่
<code>docker compose up -d --build</code>	build + เปิดทั้งระบบเบื้องหลัง
<code>docker compose ps</code>	สถานะโปรเจกต์นี้ — มองหา <code>(healthy)</code>
<code>docker compose logs vote --tail 20</code>	log รายบริการ
<code>docker compose exec redis redis-cli</code>	รันคำสั่งใน service
<code>docker compose stop</code>	หยุดเฉย ๆ — <code>start</code> กลับได้ทันที
<code>docker compose down</code>	ลบ container + network — volume สอด



คะแนนโหวตรอดตายไหม?

โหวต 6 : 4 แล้วลบทั้งระบบทิ้ง...

down แล้ว up ใหม่

/data → {"cats": 6, "dogs": 4} 🎉

คะแนนอยู่ครบ — volume สอดจาก down

down -v แล้ว up ใหม่

Volume ..._vote-data **Removed**

/data → {"cats": 0, "dogs": 0} — ไร้ความทรงจำ

container ตายได้ ข้อมูลต้องอยู่ — down ปลอดภัย · -v คือตั้งใจลบ



ระบบจริงที่รันขึ้นมา — สองหน้าเว็บ สอง service



: 8085 — vote : กดปุ่มแล้วคะแนน cats +1 ใน redis



: 8086 — result : บาร์ขยับสดทุก ~2 วิ

ทั้งคู่ต่อ redis ด้วยโค้ด Redis (host="redis") — ชื่อ service ใช้ได้เพราะ Embedded DNS ที่ compose สร้างให้ (ความรู้ Section 4 ใช้จริงที่นี่)



ชะตากรรมของข้อมูล

- 1 ปิดงานวันนี้ พรุ่งนี้ทำต่อ — stop / down / down -v?
- 2 ลบ container redis ทั้งตัว ทำไมคะแนนยังกลับมา?

 **คลิกเพื่อดูเฉลย**

1) down (หรือ stop) — ห้าม -v เพราะลบ volume · **2)** คะแนนอยู่ใน volume ไม่ใช่ตัว container — redis ใหม่ mount ลูกเดิม อ่านข้อมูลที่บันทึกไว้คืน



Error ที่เจอบ่อยใน section นี้

อาการ	สาเหตุ + ทางแก้
<code>no configuration file provided: not found</code>	อยู่นอกโฟลเดอร์ compose — <code>cd</code> ก่อน
<code>up</code> ค้างที่ <code>redis-1 Waiting</code>	ไม่ผ่าน healthcheck — อ่าน <code>compose logs redis</code>
แก้โค้ดแล้วหน้าเว็บไม่เปลี่ยน	รัน image เก่า — <code>up -d --build</code> (หรือ <code>--watch</code> — Section 6)



ประกอบแอปโหวตทั้งระบบด้วยคำสั่งเดียว

005_LAB_Docker_Compose_Voting · ~20 นาที · verify.sh

- ▶ อ่าน `docker-compose.yml` ที่ละ key — จับคู่ flag เดิม
- ▶ `up -d --build` — ดู **redis Healthy** ก่อนเว็บเปิด
- ▶ โหวต 2 แท็บ · แอบดู Redis ด้วย `compose exec`
- ▶ `down` vs `down -v` — คะแนน 6:4 รอดหรือหาย
- ▶ โบนัส `up --watch` · ปิดท้าย `down -v --rm` local



| สรุป Section 5

- ▶ ทิ้งระบบ = ไฟล์ YAML เดียว — commit ลง git ได้
- ▶ up/down คำสั่งเดียว — compose เรียงลำดับให้เอง
- ▶ healthcheck + service_healthy = รอ "พร้อมจริง" ไม่ใช่แค่ start
- ▶ ชื่อ service = ชื่อ DNS · volume ทำให้ข้อมูลรอดข้ามการลบ

ความรู้ครบทุกชิ้นแล้ว — Section 6 : เทคนิคทันสมัยที่วงการใช้จริงทุกวัน



06

อัปเดตสกิล — ของทันสมัยที่ควรรู้

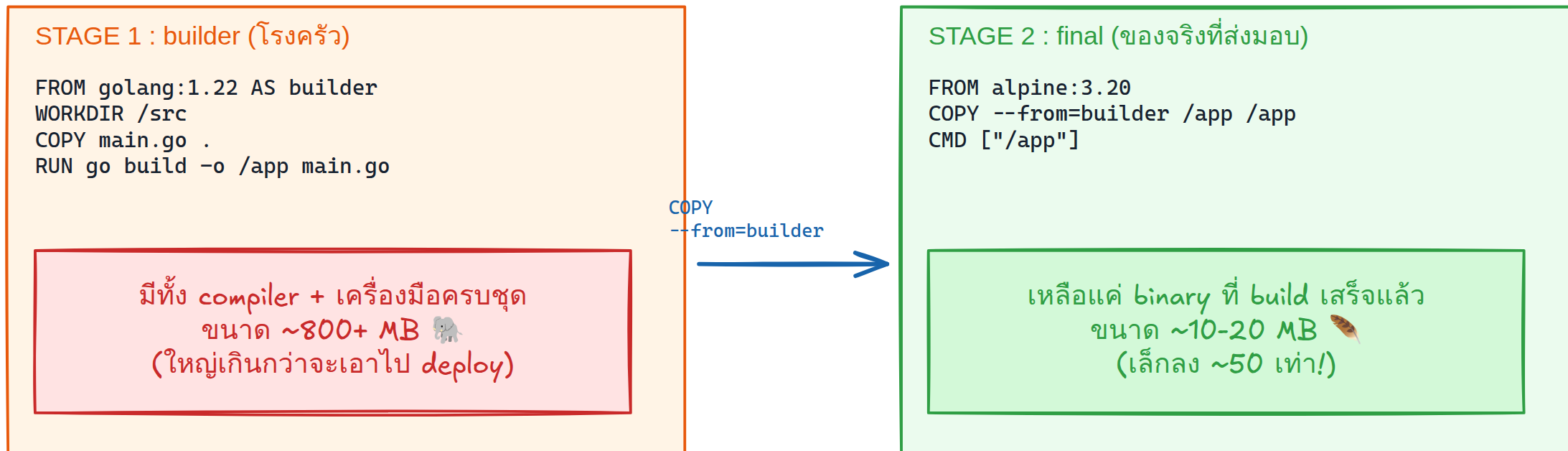
multi-stage build · compose watch · นิสัยดีของคนเขียน Dockerfile



Multi-stage Build — โรงครัวใหญ่ เสิร์ฟจานเล็ก

ปัญหา : image พก compiler ไปด้วยจนใหญ่ · ทางแก้ : คลาย FROM ในไฟล์เดียว แล้ว output ปกติใช้ stage สุดท้ายที่มีเฉพาะไฟล์จำเป็น

Multi-stage Build — โรงครัวใหญ่ เสิร์ฟจานเล็ก



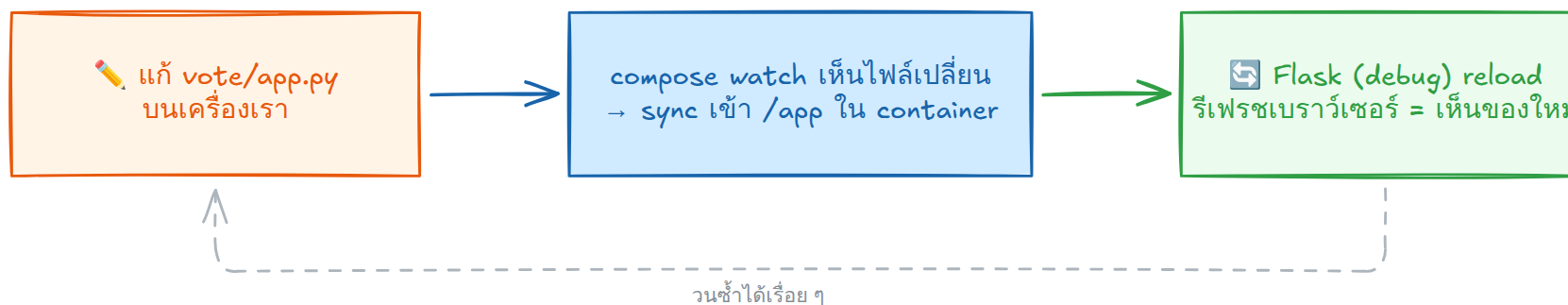
ทำไมสำคัญ : image เล็ก = pull/deploy เร็ว · พื้นที่โจมตี (attack surface) น้อยลง · ไม่ชอบ compiler ไป production
ใช้ได้กับทุกภาษา — Python ก็ใช้แยก stage ติดตั้ง dependencies ออกจาก runtime ได้เช่นกัน



| docker compose up --watch — วงจร dev ยุคใหม่

ทดลองจริงในโบนัส LAB 5 — แก้โค้ดแล้วหน้าเว็บเปลี่ยนเองใน ~2 วินาที

docker compose up --watch — แก้โค้ดแล้วเห็นผลทันที (ของใหม่ทันสมัย!)



```
vote:
  build: ./vote
  develop:
    watch:
      - action: sync
        path: ./vote
        target: /app
```

ทำไมเจ๋ง?

เมื่อก่อน : แก้โค้ด → build ใหม่
→ stop → run ใหม่ (เสียเวลาเป็นนาที)
ตอนนี้ : แก้โค้ด → เห็นผลใน ~2 วินาที

(action อื่น : rebuild, sync+restart)

ใช้เฉพาะตอนพัฒนา (dev) เท่านั้น — production ยัง build image ใหม่ตามปกติ · ทดลองจริงในโบนัสของ LAB 5



รอบสุดท้าย

- 1 อยากได้ image Go เล็ก ๆ ไม่มี compiler ติดไป — ใช้เทคนิคไหน?
- 2 แก้วโค้ดแล้วอยากเห็นผลใน ~2 วิ โดยไม่ build — ใช้ท่าไหน?

คลิกเพื่อดูเฉลย

1) multi-stage build — stage แรก compile, stage สุดท้าย copy เฉพาะ binary · **2)** `docker compose up --watch` + บล็อก `develop: watch: sync` โค้ดเข้า container



นิสยัติของคนเขียน Dockerfile (1/2)

1 • ระบุมเวอร์ชัน

ใช้ `python:3.12-slim` แทน `latest` .
ถ้าต้องการค่าเดิมแน่นอนให้ pin ด้วย
`@sha256:digest`

2 • เรียง layer ตามความถี่

ของนิ่ง (deps) อยู่บน ของเปลี่ยนบ่อย (โค้ด)
อยู่ล่าง — cache ทำงานเต็มที่

3 • มี `.dockerignore`

กัน `.git`, `.env`, ไฟล์ขยะ — context เล็ก
และ secret ไม่หลุด



นิสยติของคนเขียน Dockerfile (2/2)

4 · อย่ารันเป็น root

สร้างผู้ใช้ตาม base image แล้วปิดท้าย
USER app — ลดสิทธิ์ของ process ใน
container

5 · ห้ามฝัง secret ใน image

ส่งผ่าน -e / secret manager — image ถูก
แกะดูได้เสมอ

6 · ใส่ healthcheck

ตอบได้ว่า "พร้อมจริงไหม" — คู่
service_healthy



5 LAB พิสูจน์อะไรบ้าง

LAB	หลักฐาน	ต้องจำ
1	<code>CACHED</code> เร็ว 4 เท่า	ของเปลี่ยนบ่อยไว้ล่าง
2	<code>date</code> เป็น ASCII	CMD แทนได้ · ENTRYPOINT ต่อ
3	push ผิดชื่อ = denied	<code>registry/user/repo:tag</code>
4	ชื่อได้ในวงเรา	Embedded DNS <code>127.0.0.11</code>
5	6:4 รอด <code>down</code> หายกับ <code>-v</code>	ทั้งระบบในไฟล์เดียว

ผู้ใช้ image → ผู้สร้างระบบ 🎓



ล้างกระดานหลังเลิกเรียน 🧹

ท้ายทุกแล็บมี cleanup ของตัวเองแล้ว — ชุดนี้คือ "ล้างใหญ่" ให้เครื่องเรียนกลับเป็นศูนย์

```
# หยุดและลบ container ทั้งหมด
docker stop $(docker ps -a -q) 2>/dev/null || true
docker rm $(docker ps -a -q) 2>/dev/null || true

# ลบ image / volume / network ที่ไม่ใช้แล้ว
docker rmi -f $(docker images -q) 2>/dev/null || true
docker volume rm $(docker volume ls -q) 2>/dev/null || true
docker network prune -f

# โปรเจกต์ compose : ลบครบจบในคำสั่งเดียว (รันในโฟลเดอร์โปรเจกต์)
docker compose down -v --rmi local
```

ใช้ในเครื่องเรียนเท่านั้น! ชุดนี้กวาดเรียบทั้งเครื่อง (ทุก container/image/volume) — เครื่องจริงให้ลบเป็นรายชื่อ



DOCKER WEEK 11 — จบบริบูรณ์

Dockerfile → Registry → Network → Compose

การบ้านคิดต่อ : แอปที่คุณเคยเขียน ถ้าประกาศเป็นไฟล์ compose จะมีที่ service?
อะไรควรมี volume? อะไรควรอยู่หลัง back-tier?

เส้นทางถัดไป : **CI/CD build image อัตโนมัติ** · **Kubernetes จัดวงข้ามหลายเครื่อง**